



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

**OCENA JAKOŚCI I WYDAJNOŚCI GENERATORA LICZB
LOSOWYCH NEOTRNG W UKŁADZIE FPGA W ZALEŻNOŚCI OD
KONFIGURACJI OSCYLATORÓW PIERŚCIENIOWYCH ORAZ
ZASTOSOWANYCH EKSTRAKTORÓW LOSOWOŚCI**

Szymon Krzysztof Gogulski, 147403

Promotor
dr inż. Michał Melosik

POZNAŃ 2026

Spis treści

1	Wstęp	1
1.1	Znaczenie losowości w systemach brzegowych	1
1.2	Cel pracy	2
1.3	Układ pracy	3
2	Teoretyczne aspekty projektowania sprzętowych generatorów liczb losowych	4
2.1	Źródła losowości	4
2.1.1	TRNG	4
2.1.2	PRNG	5
2.1.3	CSPRNG	6
2.1.4	QRNG	6
2.2	RO jako źródło losowości w FPGA	7
2.3	Ekstraktory losowości	8
2.3.1	Ekstraktor von Neumanna	8
2.3.2	Wielomian CRC	9
2.3.3	Ekstraktor Toeplitza	9
2.3.4	Szyfr strumieniowy AES-CTR	9
2.4	Metody oceny losowości	9
2.4.1	Program ENT	9
2.4.2	NIST SP 800-22 – Zestaw testów statystycznych dla generatorów liczb losowych i pseudolosowych do zastosowań kryptograficznych	10
2.4.3	NIST SP 800-90B – Zalecenia dotyczące źródeł entropii używanych do generowania losowych bitów	11
3	Modułowa platforma sprzętowa do oceny losowości generatora neoTRNG	12
3.1	Architektura bazowa neoTRNG opartego na RO	12
3.1.1	Modyfikacje wprowadzone w module neoTRNG	13
3.1.2	Opis pracy komórki entropii neoTRNG_cell	14
3.2	Projekt i implementacja macierzowego ekstraktora algebraicznego Toeplitza	15
3.3	Projekt i implementacja ekstraktora kryptograficznego AES-CTR	19
3.4	Parzysta liczba inwerterów	21
3.5	Wykorzystanie zasobów sprzętowych matrycy FPGA	22
4	Badania eksperymentalne	24
4.1	Wpływ obecności post-processingu oraz liczby oscylatorów RO na losowość	24
4.1.1	NIST SP 800-22	24
4.1.2	NIST SP 800-90B	26
4.1.3	ENT	26

4.2	Wpływ rozmieszczenia RO w układzie FPGA na losowość	26
4.2.1	NIST SP 800-22	28
4.2.2	NIST SP 800-90B	31
4.2.3	ENT	31
4.3	Wpływ ekstraktorów AES-CTR i Toeplitz na losowość przy zredukowanej liczbie RO	32
4.3.1	NIST SP 800-22	32
4.3.2	NIST SP 800-90B	33
4.3.3	ENT	33
4.4	Wydajność neoTRNG w zależności od typu ekstrakcji	34
5	Podsumowanie i wnioski	36
5.1	Wpływ liczby oscylatorów i liczby inwerterów	36
5.2	Efektywność przetwarzania końcowego von Neumanna oraz CRC	36
5.3	Wpływ rozmieszczenia RO	37
5.4	Zastosowanie zaawansowanych metod przetwarzania końcowego	37
5.5	Podsumowanie końcowe	37
	Literatura	39
	A Floorplanning	41
	B Automatyzacja pomiarów i testów	43
B.1	Wymagania sprzętowe	43
B.2	Zbieranie danych	43
B.3	Uruchamianie pojedynczych testów	44
B.4	Struktura zbioru danych	46
B.5	Automatyzacja testów	47

Spis rysunków

2.1	Model źródła entropii. [19]	5
2.2	Quantis USB 4M. [12]	6
2.3	Model układu optycznego. [3]	7
2.4	Przykład prostego oscylatora pierścieniowego.	7
2.5	Propagacja sygnału wewnątrz inwertera CMOS – symulacja. [13]	8
2.6	Przykładowy wynik pakietu ENT.	10
3.1	Architektura neoTRNG [18].	13
3.2	Parametry generic oraz porty neoTRNG.	13
3.3	Parametry generic oraz porty neoTRNG_cell.	14
3.4	Metastabilność – stan nieustalony [10].	15
3.5	Wynik symulacji ekstraktora Toeplitza.	16
3.6	Diagram ideowy modułu neoTRNG zintegrowanego z ekstraktorem Toeplitza.	17

3.7	Schemat generatora z ekstraktorem Toeplitza.	18
3.8	Wyniki symulacji silnika AES.	19
3.9	Architektura modułu nadrzędnego AES-CTR wyposażonego w maszynę stanów, generator neoTRNG, silnik AES, rejestr licznika oraz moduł komunikacji.	20
3.10	Diagram stanów kontrolera pracy generatora z silnikiem AES-CTR.	21
3.11	Generator z parzystą liczbą RO.	22
3.12	Ciągły sygnał '1' z generatora dla parzystej liczby inwerterów.	22
3.13	Ciągły sygnał '0' z generatora dla parzystej liczby inwerterów.	22
4.1	2 RO rozproszone.	27
4.2	2 RO zlokalizowane.	27
4.3	3 RO rozproszone.	27
4.4	3 RO zlokalizowane.	27
4.5	6 RO rozproszone.	27
4.6	6 RO zlokalizowane.	27
A.1	Okno dialogowe po prawej stronie	41
A.2	Ręczne wyznaczanie granic Pblock	42
A.3	Wybór elementów układu mających znaleźć się w obrębie Pblock	42
B.1	Aparatura pomiarowa	43
B.2	Dialog NIST SP 800-22.	45
B.3	Struktura katalogów dla konfiguracji 2RO_8I.	46
B.4	Uruchomienie skryptu automatyzacji testów ENT dla /1_testy_bazowe/2RO_8I. . .	47

Spis tabel

2.1	Specyfikacja ogólna urządzenia Quantis-USB-4M [12]	6
2.2	Opis 15 testów statystycznych pakietu NIST SP 800-22 Rev. 1a [14]	10
3.1	Wpływ konfiguracji parametrów generycznych na rzeczywisty sygnał wyjściowy generatora	14
3.2	Compiled Resource Utilization Results across Chapters	23
4.1	NIST 800-22 : 2 RO : 3-5	25
4.2	NIST 800-22 : 3 RO : 3-5-7	25
4.3	NIST 800-22 : 3 RO : 5-7-9	26
4.4	NIST 800-90 : 3 RO 5-7-9 / 3 RO 3-5-7 / 2 RO 3-5	26
4.5	ENT : 3 RO 5-7-9 / 3 RO 3-5-7 / 2 RO 3-5	26
4.6	NIST 800-22 : 2 RO : 3-5 rozproszone	28
4.7	NIST 800-22 : 2 RO : 3-5 zlokalizowane	29
4.8	NIST 800-22 : 3 RO : 5-7-9 rozproszone	29
4.9	NIST 800-22 : 3 RO : 5-7-9 zlokalizowane	30

4.10	NIST 800-22 : 6 RO : 5-7-9-11-13-15 rozproszone	30
4.11	NIST 800-22 : 6 RO : 5-7-9-11-13-15 zlokalizowane	31
4.12	NIST 800-90 : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 rozproszone	31
4.13	NIST 800-90 : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 zlokalizowane	31
4.14	ENT : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 rozproszone	32
4.15	ENT : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 zlokalizowane	32
4.16	NIST 800-22 : AES 3RO 3-5-7 / Toeplitz 3RO 3-5-7 / AES 2RO 3-5 / Toeplitz 2RO 3-5	33
4.17	NIST 800-90 : AES 3RO 3-5-7 / Toeplitz 3RO 3-5-7 / AES 2RO 3-5 / Toeplitz 2RO 3-5	33
4.18	ENT : AES 3RO 3-5-7 / Toeplitz 3RO 3-5-7 / AES 2RO 3-5 / Toeplitz 2RO 3-5 . . .	33
4.19	Porównanie przepustowości dla różnych konfiguracji oraz metod post-processingu. . . .	34

Rozdział 1

Wstęp

1.1 Znaczenie losowości w systemach brzegowych

Rosnąca złożoność systemów informatycznych oraz konieczność przesyłania przez sieć coraz większych wolumenów danych wymuszają na inżynierach poszukiwanie nowych rozwiązań architektonicznych. Odpowiedzią na te wyzwania i uniezależnienie współczesnych systemów komputerowych od centralnych usług chmurowych jest rozwój paradygmatu przetwarzania brzegowego (ang. *edge computing*). Przetwarzanie brzegowe polega na przeniesieniu procesów obliczeniowych z chmury na dedykowane urządzenia znajdujące się na krawędzi sieci [15]. W tym podejściu zaawansowane etapy analizy danych i podejmowania decyzji realizowane są lokalnie, bezpośrednio w miejscu ich generowania, zanim ewentualne wyniki zostaną przesłane do chmury.

Choć model ten skutecznie ogranicza zależność od scentralizowanej infrastruktury i odciąża sieć, wprowadza on jednocześnie nowe, specyficzne dla środowisk rozproszonych wektory ataków i zagrożenia bezpieczeństwa. W warstwie sprzętowej architektury systemów brzegowych często opierają się na wykorzystaniu układów FPGA, komputerów przemysłowych lub dedykowanych układów scalonych (ang. ASIC – *Application-Specific Integrated Circuit*). Implementacja z wykorzystaniem układów FPGA, w porównaniu do układów ASIC, zapewnia pełną rekonfigurowalność sprzętową, jednak odbywa się to kosztem stosunkowo wysokiego poboru mocy. Odwrotna sytuacja ma miejsce w przypadku układów ASIC, które po wyprodukowaniu nie dają możliwości rekonfiguracji, ale charakteryzują się minimalnym zapotrzebowaniem na energię. Główną zaletą obu tych architektur jest możliwość ścisłego dostosowania ich do specyfiki realizowanych zadań. Osiągnięcie tej optymalizacji wiąże się jednak z wysokim stopniem skomplikowania procesu projektowego, co otwiera system na nowe podatności. Jedną ze szczególnych klas zagrożeń, bezpośrednio wynikających ze złożoności cyklu rozwoju i rozproszonego łańcucha dostaw, są trojany sprzętowe (ang. *Hardware Trojans*). Atakujący mogą je zaimplementować na etapie projektowania lub fizycznej produkcji w przypadku układów ASIC albo wprowadzić do źródeł projektowych (np. na poziomie kodu HDL) w układach FPGA.

Ze względu na istotność tego typu zagrożenia przyjęto, że jedną z metod przeciwdziałania jest promowanie układów scalonych rozwijanych w sposób transparentny i weryfikowalny. Organizacje takie jak OSHWA (ang. *Open-Source Hardware Association*) i projekty takie jak RISC-V stanowią fundament ruchu wspierającego transparentny rozwój otwartoźródłowych rozwiązań technicznych. OSHWA oraz architektura zestawu instrukcji RISC-V stanowią odpowiedź na kryzys zaufania do zamkniętych i komercyjnych architektur. Znajdują się we wspólnym ekosystemie otwartoźródłowych technologii, choć pełnią różne funkcje. Architektura RISC-V jest jedynie standardem technicznym, który umożliwia rozwój systemów wbudowanych i tym samym samych układów scalonych.

Z kolei OSHWA jest organem nadzorującym rozpowszechnianie konkretnych, gotowych rozwiązań. Połączenie projektu i organizacji poskutkowało powstaniem szerokiego wachlarza transparentnych, otwartoźródłowych i bezpiecznych projektów układów scalonych.

Systemy brzegowe, oprócz jednostek dedykowanych do przetwarzania i transmisji danych, posiadają w swojej architekturze zintegrowane moduły kryptograficzne. W kontekście OSHWA budowane są kompletne systemy SoC (ang. *System-on-a-Chip*), zawierające niezbędne urządzenia peryferyjne wspierające bezpieczeństwo. Jednymi z najistotniejszych komponentów wykorzystywanych w procesach kryptograficznych są generatory liczb losowych [17].

Liczby losowe wykorzystywane są na różnych etapach procesu kryptograficznego. Bezpieczeństwo komunikacji opartej na algorytmie RSA bazuje na wysokiej złożoności obliczeniowej problemu faktoryzacji iloczynu dwóch dużych liczb pierwszych [21]. W praktycznych implementacjach sprzętowych i programowych proces wyznaczania tych liczb rozpoczyna się od pobrania wartości z generatora losowego. Na tej podstawie konstruowane są duże liczby całkowite, które następnie poddawane są testom pierwszości. Bezpieczeństwo całej pary kluczy zależy więc bezpośrednio od jakości zastosowanego źródła losowości. Jeżeli wyjściowy ciąg bitów z generatora jest znany lub kontrolowany, wygenerowane na jego podstawie liczby pierwsze stają się częściowo przewidywalne. W takiej sytuacji złożoność problemu faktoryzacji drastycznie spada, czyniąc atak obliczeniowo opłacalnym i prowadząc do całkowitej kompromitacji klucza prywatnego.

Tworzenie podpisów cyfrowych nie byłoby możliwe bez odpowiednich źródeł losowości. Kryptografia krzywych eliptycznych, stosowana przy podpisywaniu plików binarnych, zawiera element losowy w postaci jednorazowej wartości nonce. Sytuacja, w której nonce nie spełnia miar losowości, naraża na niebezpieczeństwo cały klucz prywatny producenta. Gdy kilka kolejnych bitów losowych wartości nonce (używanych przy każdej generacji sygnatury) jest znanych dla pewnej liczby sygnatur cyfrowych, możliwe jest odzyskanie klucza prywatnego [9].

W różnych trybach pracy szyfrów blokowych, takich jak AES, wprowadzane są dodatkowe wartości losowe, takie jak wektor inicjalizacyjny (IV) czy nonce. Powtarzająca się wartość nonce w trybie AES-CTR naraża na niebezpieczeństwo całą szyfrowaną wiadomość. Przechwycenie dwóch różnych szyfrogramów wygenerowanych z tym samym nonce umożliwia wydobycie tekstu jawnego [2].

Wiarygodność sprzętowych generatorów liczb losowych, rozumiana jako implementacja wolna od trojanów sprzętowych, może być zapewniona przez rozwój tego typu układów w koncepcji OSHWA. Projekt „The NEORV32 RISC-V Processor” jest jedną z realizacji idei OSHWA i RISC-V. Jest to otwartoźródłowa, modyfikowalna platforma sprzętowa napisana w języku opisu sprzętu VHDL. Rozwiązanie to może być dedykowane inżynierom rozwijającym systemy brzegowe w oparciu o układy FPGA. W strukturze procesora NEORV32 RISC-V zintegrowano moduł generatora entropii neoTRNG, który stanowi główny przedmiot badań prowadzonych w ramach niniejszej pracy magisterskiej.

1.2 Cel pracy

Celem niniejszej pracy magisterskiej jest adaptacja oraz implementacja otwartoźródłowego generatora losowości neoTRNG, bazującego na oscylatorach pierścieniowych. Badany moduł będzie implementowany jako układ cyfrowy realizowany na płycie deweloperskiej wyposażonej w układ FPGA. Badania zostaną przeprowadzone za pomocą dedykowanego środowiska testowego.

Kryteria oceny generatora obejmować będą: zestaw testów statystycznych zdefiniowanych w publikacji NIST SP 800-22 [14], zestaw testów do oceny entropii opisany w publikacji NIST SP 800-90B [19] oraz program do oceny generatorów pseudolosowych ENT [4]. Wraz z wynikami te-

stów zestawione zostaną informacje o zajętości zasobów sprzętowych i przepustowości poszczególnych implementacji. Badane moduły będą stanowiły modyfikacje bazowego rozwiązania neoTRNG przedstawionego w repozytorium [18]. Modyfikacje obejmą aspekty takie jak: liczba zastosowanych oscylatorów, zastosowanie korekcji źródła (np. korektor Von Neumanna), metody przetwarzania końcowego (ang. *post-processing*) będące ekstraktorami losowości ze źródła entropii oraz fizyczne rozmieszczenie elementów w strukturze FPGA.

1.3 Układ pracy

Struktura pracy obejmuje pięć rozdziałów oraz dodatkową sekcję załączników.

Rozdział pierwszy przedstawia motywację podjęcia badań w obszarze generatorów losowości oraz określa założenia i cele pracy.

Rozdział drugi koncentruje się na wybranych podstawach teoretycznych. Omówiono w nim definicje związane z tematyką pracy, dostępne ekstraktory entropii oraz metody oceny losowości.

Rozdział trzeci omawia architekturę badanego modułu neoTRNG oraz przedstawia własne (autorskie) modyfikacje badanego generatora wraz z analizą ich zajętości sprzętowej.

Rozdział czwarty poświęcono badaniom eksperymentalnym. Zbadane zostaną:

- wpływ skalowania liczby oscylatorów pierścieniowych oraz liczby inwerterów na losowość,
- wpływ zastosowania ekstraktorów AES-CTR i Toeplitza na losowość,
- wpływ rozmieszczenia komórek entropii w układzie FPGA na losowość.

Wraz z wynikami testów losowości dołączona zostanie informacja o przepustowości (wydajności uzyskiwanego ciągu bitów)

Rozdział piąty kończy pracę i podsumowuje najważniejsze wnioski wynikające z przeprowadzonych badań.

Załącznik A zawiera instrukcję planowania rozmieszczenia elementów (ang. *floorplanning*), wykonanego w ramach analizy rozmieszczenia komórek entropii.

Załącznik B zawiera instrukcję przedstawiającą metodę akwizycji oraz przetworzenia danych testowych.

Rozdział 2

Teoretyczne aspekty projektowania sprzętowych generatorów liczb losowych

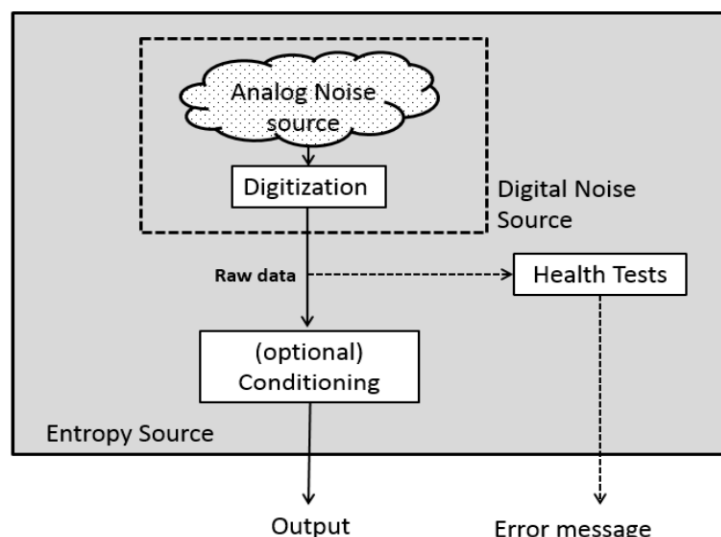
Niniejszy rozdział przedstawia teoretyczne podstawy projektowania sprzętowych generatorów liczb losowych. Omówiono w nim klasyfikację architektur generatorów oraz zjawiska fizyczne wykorzystywane do pozyskiwania entropii w układach programowalnych FPGA. Ponadto wyjaśniono metody obróbki sygnałów losowych, a także standardy statystyczne stosowane do walidacji jakości generowanych danych.

2.1 Źródła losowości

Generatory liczb losowych można podzielić na kategorie w zależności od mechanizmu powstawania losowości. Dwie główne klasy generatorów stanowią generatory liczb prawdziwie losowych (ang. *TRNG* – *True Random Number Generator*) oraz generatory liczb pseudolosowych (ang. *PRNG* – *Pseudorandom Number Generator*). Spośród generatorów liczb prawdziwie losowych można wyróżnić podklasę kwantowych generatorów liczb prawdziwie losowych (ang. *QRNG* – *Quantum Random Number Generator*) [7]. Podklasą generatorów pseudolosowych są generatory pseudolosowe kryptograficznie bezpieczne (ang. *CSPRNG* – *Cryptographically Secure Pseudorandom Number Generator*). Kryterium podziału generatorów na TRNG i PRNG jest źródło pochodzenia losowości.

2.1.1 TRNG

Generatory liczb prawdziwie losowych opierają generowanie liczb na digitalizacji niedeterministycznych zjawisk fizycznych. Przykładowymi źródłami szumów analogowych są: szum napięcia zasilania, fluktuacje temperatury elementów pasywnych, niestabilności fazowe w układach cyfrowych (oscylatory pierścieniowe) lub bardziej nietypowe zjawiska, takie jak procesy kwantowe w układach optycznych.



RYSUNEK 2.1: Model źródła entropii. [19]

Wiodącą publikacją w dziedzinie projektowania generatorów liczb prawdziwie losowych jest zbiór rekomendacji projektowych *NIST SP 800-90B*. Publikacja ta opisuje ogólny model źródła losowości złożony z czterech części składowych [19]:

- źródło szumu analogowego – entropia zbierana z niedeterministycznego procesu fizycznego,
- digitalizacja – próbkowanie oraz kwantyzacja sygnału analogowego poprzez np. przetwornik analogowo-cyfrowy,
- przetwarzanie końcowe – metody postprocessing’u w celu zwiększenia jakości wygenerowanej entropii,
- testy zdrowia – moduły monitorujące pracę źródła, wykrywające awarie i próby manipulacji.

Mimo wielu zalet generatory liczb prawdziwie losowych nie są całkowicie odporne na ataki. Możliwą formą ataku jest zakłócenie źródła szumu analogowego, na przykład poprzez rozgrzanie termistora próbującego fluktuacje temperatury.

2.1.2 PRNG

Generatory liczb pseudolosowych charakteryzują się generowaniem sygnału pozornie losowego. Wynika to z mechanizmów wewnątrz takiego generatora. „Generowanie liczb pseudolosowych polega na tworzeniu długich sekwencji bitów, zaczynając od małej, najlepiej prawdziwie losowej wartości początkowej zwanej ziarnem” [1]. Główną wadą takich generatorów jest przewidywalność całego ciągu losowego po poznaniu wykorzystanego ziarna. Bazują one na podstawowych operacjach algebraicznych, generując kolejne wartości na podstawie wyników powtarzających się obliczeń. Z tego względu klasyczne algorytmy PRNG (LCG, Mersenne Twister) nie są uważane za generatory bezpieczne kryptograficznie. Są one podatne na analizę sekwencji ciągu, co umożliwia przewidzenie kolejnych wygenerowanych wartości [11]. Ze względu na ich prostotę znajdują one zastosowanie w systemach niekrytycznych. Są stosowane przy generowaniu proceduralnym bądź w symulacjach naukowych.

2.1.3 CSPRNG

Kryptograficznie bezpieczne generatory liczb pseudolosowych działają w sposób analityczny jak generatory PRNG, z jedną szczególną różnicą: wykorzystują nieodwracalne funkcje kryptograficzne. Tego typu generatory muszą spełniać dwa wymagania: test następnego bitu (ang. *next-bit test*) oraz odporność na kompromitację stanu [22]. Oznacza to, że poprzez znajomość ciągu N bitów losowych atakujący nie jest w stanie przewidzieć bitu $N + 1$ z prawdopodobieństwem większym niż 50% oraz, znając część lub całość stanu generatora, nie jest w stanie odtworzyć poprzednich wygenerowanych wartości [22].

2.1.4 QRNG

Kwantowe generatory liczb losowych stanowią szczególny przypadek generatora prawdziwie losowego. Zamiast pobierać entropię ze zjawisk fizyki klasycznej, wykorzystują zjawiska mechaniki kwantowej. Ich główną przewagą jest probabilistyczna natura zjawisk kwantowych. Zjawiska fizyki klasycznej, takie jak zmiany temperatury, można zamodelować za pomocą cząstkowych równań różniczkowych, co może dać niepożądany wgląd do wnętrza mechanizmu generującego losowość. W przypadku zjawisk kwantowych modele takie nie istnieją – obserwowany układ kwantowy zawsze będzie działał z określonym rozkładem prawdopodobieństwa [8]. Wadą kwantowych generatorów jest ich cena. Zazwyczaj rozwiązania kwantowe są znacznie droższe niż generatory klasyczne.

Komercyjnie dostępny jest kwantowy generator Quantis-USB-4M firmy IDQ. Urządzenie to dostępne w obudowie z interfejsem USB 2.0.



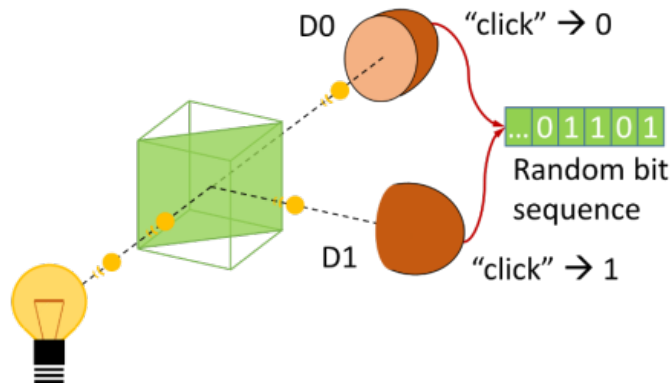
RYSUNEK 2.2: Quantis USB 4M. [12]

TABELA 2.1: Specyfikacja ogólna urządzenia Quantis-USB-4M [12]

Parametr	Specyfikacja
Losowa prędkość bitowa*	4 Mbit/s $\pm$ 10%
Udział szumu termicznego	< 1%
Temperatura przechowywania	od -25 do +85°C
Wymiary	61 mm $\times$ 31 mm $\times$ 114 mm
Specyfikacja USB	2.0
Wymagania	Komputer PC z wolnym złączem USB
Zasilanie	Przez port USB

* Przepustowość (bitrate) przed ekstrakcją losowości

Według specyfikacji producenta (Tab. 2.1) udział szumu termicznego w generowanych bitach losowych wynosi mniej niż 1%. Oznacza to, że generator wytwarza losowość niemal wyłącznie ze zjawisk mechaniki kwantowej.



RYSUNEK 2.3: Model układu optycznego. [3]

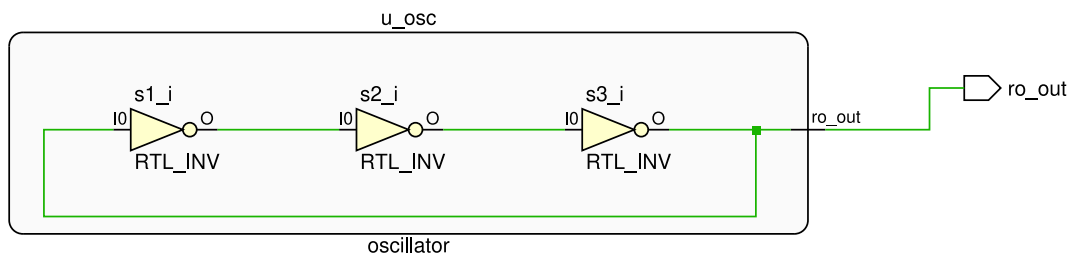
Układ optyczny znajdujący się wewnątrz Quantis USB 4M składa się z czterech elementów:

- źródła fotonów – laser o niskiej mocy generujący wiązkę fotonów,
- dzielnika wiązki – pryzmat wykonany z materiałów, które częściowo przepuszczają fotony,
- dwóch detektorów fotonów.

Generowana losowość jest związana z częściową przepuszczalnością dzielnika wiązki. W przypadku idealnym 50% fotonów zostaje odbitych, a 50% zostaje przepuszczonych. Oznacza to, że prawdopodobieństwo detekcji fotonu w każdym z detektorów jest rozłożone równo po 50%. W ten sposób generator kwantowy wytwarza ciąg bitów niedeterministycznych z jednostajnym rozkładem bitu '1' i '0'. Jednak w rzeczywistości dzielnik wiązki nie jest elementem idealnym. Oznacza to, że prawdopodobieństwo wystąpienia bitu '1' lub '0' nie będzie równe idealnie 50%.

2.2 RO jako źródło losowości w FPGA

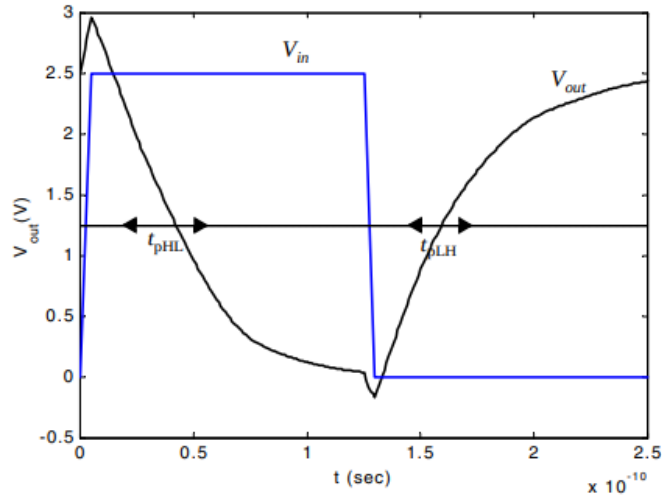
Generowanie losowości przy użyciu oscylatorów pierścieniowych bazuje na zjawiskach fizycznych związanych z propagacją sygnałów wewnątrz struktury krzemowej układu FPGA. Oscylator pierścieniowy (ang. *RO* – *Ring Oscillator*) jest asynchronicznym układem cyfrowym złożonym z pętli o nieparzystej liczbie inwerterów.



RYSUNEK 2.4: Przykład prostego oscylatora pierścieniowego.

Tak skonstruowany układ logiczny doprowadzi do powstania stanu nieustalonego wewnątrz układu. Inwertery ułożone w pętli będą kolejno generować sygnał wejściowy dla następnego inwertera w kolejności. Oznacza to, że w układzie powstanie sygnał oscylujący z okresem wynikającym z

czasu propagacji sygnału w krzemie oraz liczby zastosowanych inwerterów. Należy zwrócić uwagę, że ma to zastosowanie wyłącznie dla nieparzystej liczby inwerterów. W przypadku liczby parzystej sygnał się ustali, a próbkowane wyjście będzie zwracać wartość stałą.



RYСУNEK 2.5: Propagacja sygnału wewnątrz inwertera CMOS – symulacja. [13]

Na zamieszczonym rysunku (Fig. 2.5) przedstawiono przebiegi sygnału V_{in} oraz V_{out} dla symulacji inwertera wykonanego w technologii $0.25\mu m$. Widoczne jest opóźnienie w propagacji sygnału wyjściowego dla skokowej zmiany sygnału wejściowego. Od momentu przełączenia V_{in} do ustalenia sygnału V_{out} na poziomie 2.5 V mija około 100 ps.

W rzeczywistości każdy inwerter będzie odrobinę różnił się od reszty ze względu na niedoskonałe metody produkcyjne. Co za tym idzie, sygnał wyjściowy z RO będzie zmieniał się w jeszcze bardziej nieprzewidywalny sposób. Idąc dalej, pozwala to na stworzenie układu składającego się z wielu RO o różnej liczbie inwerterów, gdzie kombinacja ich wyjść wygeneruje porządaną losowość. Umożliwia to opracowanie generatora liczb prawdziwie losowych ze względu na analogową naturę zjawisk zachodzących wewnątrz układu RO.

2.3 Ekstraktory losowości

Wektor bitów pozyskiwany bezpośrednio ze źródła analogowego charakteryzuje się niejednostajnym rozkładem zer i jedynek oraz występowaniem korelacji. Aby poprawić losowość, sugerowaną praktyką jest odpowiednia korekcja ciągu bitów.

2.3.1 Ekstraktor von Neumanna

Ekstraktor von Neumanna jest nieskomplikowanym układem cyfrowym. Jego zadaniem jest zapewnienie jednostajnego rozkładu generowanych liczb losowych. Zasada działania tego ekstraktora opiera się na pobraniu dwóch próbek sygnału ze źródła oraz sprawdzeniu zestawu trzech instrukcji [20]:

- pobrano próbki (0; 0) lub (1; 1) → próbki zostają odrzucone,
- pobrano próbki (1; 0) → generowany jest bit 1,

- pobrano próbki $(0; 1) \rightarrow$ generowany jest bit 0.

Ekstraktor zapewnia jednostajny rozkład prawdopodobieństwa wystąpienia każdego z bitów, ale znacznie ogranicza wydajność generatora. Jednocześnie nie pozbywa się on korelacji z generowanego sygnału.

2.3.2 Wielomian CRC

Cykliczna kontrola nadmiarowa (ang. *CRC – Cyclic Redundancy Check*) jest pierwotnie stosowana w celu zapewnienia integralności przesyłanych danych w interfejsach komunikacyjnych [5]. Znajduje jednak również zastosowanie w korekcji źródła losowości. Zaimplementowana jest w postaci liniowego rejestru przesuwającego ze sprzężeniem zwrotnym (LFSR). Posiada właściwości rozpraszające, co oznacza, że każdy nowy bit ma wpływ jednocześnie na wszystkie bity wyjściowe [18]. W ten sposób eliminuje korelacje między wyjściowymi bitami.

2.3.3 Ekstraktor Toeplitza

Ekstraktor oparty na macierzach Toeplitza należy do grupy silnych ekstraktorów losowości (ang. *strong extractors*). Ekstrakcja polega na pomnożeniu wektora surowych bitów wejściowych X o długości n przez macierz Toeplitza M o wymiarach $m \times n$ (gdzie $m < n$):

$$Y = M \cdot X \quad (2.1)$$

Macierz Toeplitza ma tę właściwość, że każdy jej ukośny rząd ma stałą wartość, co pozwala na jej jednoznaczne zdefiniowanie za pomocą jedynie $n + m - 1$ bitów ziarna [23].

2.3.4 Szyfr strumieniowy AES-CTR

Szyfr blokowy AES działający w trybie licznika jest jednym z zalecanych systemów generowania liczb losowych. Konfiguracja AES-CTR polega na ciągłym generowaniu losowego szyfrogramu na podstawie wstępnej wartości ziarna pobranej ze źródła losowości. Na początku pracy AES-CTR pobierane są losowe wartości klucza oraz nonce. Tekst jawny jest połączeniem nonce oraz wartości z licznika. Taka konfiguracja pozwala wygenerować spore ilości danych losowych dla kolejnych wartości z licznika przy niewielkim wykorzystaniu źródła losowości. Wadą tego podejścia jest cykliczna konieczność odświeżania ziarna wykorzystanego na początku procesu [16].

2.4 Metody oceny losowości

Ocena właściwości statystycznych generatora liczb losowych złożona jest z zestawu miar i testów. W ramach badań wykorzystuje się publicznie dostępne programy do oceny generowanych ciągów losowych.

2.4.1 Program ENT

„Program ENT przeprowadza różne testy na sekwencjach bajtów zapisanych w plikach i raportuje wyniki tych testów. Program jest przydatny do oceny generatorów liczb pseudolosowych w zastosowaniach szyfrowania i próbkowania statystycznego, algorytmach kompresji i innych zastosowaniach, w których istotna jest gęstość informacji w pliku” [4].

```

1 Entropy = 7.168778 bits per byte.
2
3 Optimum compression would reduce the size
4 of this 104857600 byte file by 10 percent.
5
6 Chi square distribution for 104857600 samples is 131655940.14, and randomly
7 would exceed this value less than 0.01 percent of the times.
8
9 Arithmetic mean value of data bytes is 127.4529 (127.5 = random).
10 Monte Carlo value for Pi is 3.458389109 (error 10.08 percent).
11 Serial correlation coefficient is 0.000098 (totally uncorrelated = 0.0).

```

RYSUNEK 2.6: Przykładowy wynik pakietu ENT.

2.4.2 NIST SP 800-22 – Zestaw testów statystycznych dla generatorów liczb losowych i pseudolosowych do zastosowań kryptograficznych

Zestaw testowy NIST SP 800-22 jest standardowym kryterium oceny generatorów liczb losowych. Składa się z piętnastu testów odpowiedzialnych za sprawdzenie różnych aspektów wygenerowanego ciągu bitów.

TABELA 2.2: Opis 15 testów statystycznych pakietu NIST SP 800-22 Rev. 1a [14]

Lp.	Nazwa testu (ang.)	Krótki opis i cel testu
1	Frequency (Monobit) Test	Badanie proporcji jedynek do zer w całym ciągu binarnym. Sprawdza, czy ich liczba jest w przybliżeniu równa.
2	Frequency Test within a Block	Badanie proporcji jedynek do zer wewnątrz mniejszych, określonych bloków bitów wyciętych z ciągu.
3	Runs Test	Badanie całkowitej liczby serii (nieprzerwanych ciągów samych jedynek lub samych zer) o różnych długościach.
4	Test for the Longest Run of Ones in a Block	Test bada długość najdłuższej serii jedynek w blokach M -bitowych i sprawdza, czy jest ona zgodna z wartością oczekiwaną dla ciągu losowego.
5	Binary Matrix Rank Test	Test wyznacza rząd rozłącznych podmacierzy binarnych, aby sprawdzić liniową zależność między podciągami o stałej długości.
6	Discrete Fourier Transform (Spectral) Test	Test analizuje wysokość szczytów transformaty Fouriera (DFT) ciągu, aby wykryć bliskie, powtarzające się wzorce okresowe przeczące losowości ciągu.
7	Non-overlapping Template Matching Test	Zliczanie wystąpień określonych, nienakładających się wzorców bitowych (szablonów) w celu wykrycia anomalii struktur.
8	Overlapping Template Matching Test	Zliczanie wystąpień określonych wzorców bitowych, które mogą się na siebie nakładać (np. szukanie ciągów samych jedynek).
9	Maurer's "Universal Statistical" Test	Ocena stopnia kompresji ciągu bez utraty informacji. Sprawdza, czy ciąg można znacznie skrócić.
10	Linear Complexity Test	Badanie długości rejestru przesuwającego z liniowym sprzężeniem zwrotnym (LFSR). Sprawdza przewidywalność ciągu.
11	Serial Test	Badanie częstotliwości występowania wszystkich możliwych n -bitowych nakładających się wzorców w całym ciągu.
12	Approximate Entropy Test	Test bada częstotliwość występowania nakładających się wzorców o długościach m i $m + 1$, porównując ją z rozkładem oczekiwanym dla ciągu losowego.
13	Cumulative Sums (Cusum) Test	Analiza maksymalnego odchylenia losowego błędzenia wyznaczonego przez skumulowane sumy przekształconych bitów.
14	Random Excursions Test	Badanie liczby cykli (odwiedzin określonego stanu przed powrotem do zera) w losowym błędzeniu ciągu.
15	Random Excursions Variant Test	Wariant testu Random Excursions, sprawdzający dokładną liczbę powtórzeń konkretnych stanów w błędzeniu losowym.

2.4.3 NIST SP 800-90B – Zalecenia dotyczące źródeł entropii używanych do generowania losowych bitów

NIST SP 800-90B to publikacja stanowiąca zbiór rekomendacji związanych z projektowaniem źródeł entropii. Równocześnie razem z publikacją dostępny jest program do oceny entropii minimalnej badanego źródła. W skład publikacji wchodzi zalecenia odnośnie:

- procesu walidacji źródła,
- testów zdrowia i monitorowania źródła,
- oceny, czy próbki danych są niezależne oraz o jednakowym rozkładzie prawdopodobieństwa,
- estymacji H_{min} .

Na serię standardów NIST SP 800-90 składają się dwa kolejne dokumenty: SP 800-90A - rekomendacje dla deterministycznych generatorów liczb losowych oraz SP 800-90C - rekomendacje dla konstrukcji generatorów liczb losowych.

Rozdział 3

Modułowa platforma sprzętowa do oceny losowości generatora neoTRNG

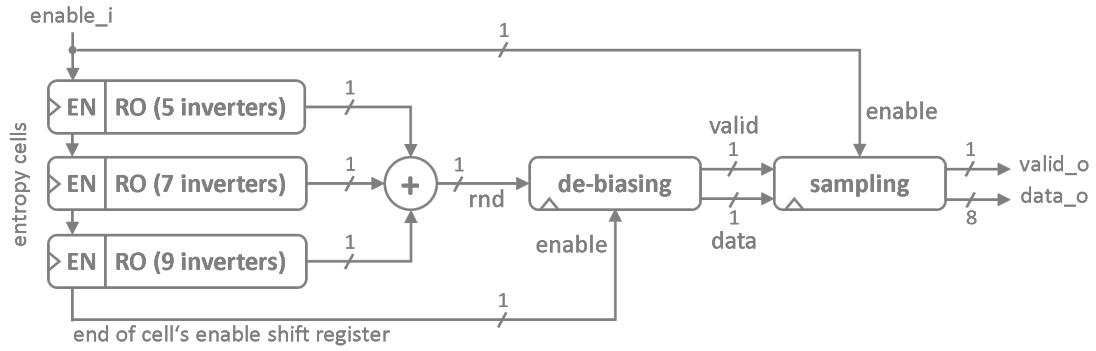
Generator liczb losowych neoTRNG stanowi obiekt badań niniejszej pracy. Cały projekt generatora jest publicznie dostępny w postaci repozytorium kodu (<https://github.com/stnolting/neoTRNG>). Główne cechy charakterystyczne badanego generatora to [18]:

- projekt niezależny od technologii konkretnego dostawcy,
- niewielkie zużycie zasobów sprzętowych,
- wysoka przepustowość,
- projekt otwartoźródłowy w postaci jednego pliku VHDL,
- wysoka częstotliwość pracy,
- łatwość integracji,
- pełna dokumentacja.

Ostrzeżenie: Domyślna implementacja neoTRNG nie gwarantuje generowania całkowicie losowych ani bezpiecznych kryptograficznie ciągów liczb. Ponadto generator nie jest wyposażony w żadne mechanizmy testów zdrowia źródła [18].

3.1 Architektura bazowa neoTRNG opartego na RO

W domyślnej konfiguracji generator jest wyposażony w trzy oscylatory pierścieniowe, mające kolejno pięć, siedem i dziewięć inwerterów. Sygnały wyjściowe z RO są łączone operacją XOR. Następnie surowy sygnał losowy trafia do ekstraktora von Neumanna. Ostatecznie próbki z ekstraktora są mieszane za pomocą wielomianu CRC. Generator zwraca jeden bajt danych, o czym informuje stanem wysokim na linii `valid_o`.



RYSUNEK 3.1: Architektura neoTRNG [18].

Bazowy plik RTL zawiera cztery parametry generyczne:

- NUM_CELLS – liczba zastosowanych RO,
- NUM_INV_START – nieparzysta liczba inwerterów w pierwszym RO (każdy kolejny RO zawiera o dwa inwertery więcej),
- NUM_RAW_BITS – liczba bitów wejściowych dla wielomianu CRC,
- SIM_MODE – dodatkowy tryb pracy, wykorzystywany do symulacji układu oraz testów (nie generuje sygnału losowego).

```

1  entity neoTRNG is
2      generic (
3          NUM_CELLS           : natural range 1 to 255;
4          NUM_INV_START       : natural range 3 to 4095;
5          NUM_RAW_BITS        : natural range 1 to 4096;
6          SIM_MODE            : boolean;
7          ENABLE_VON_NEUMANN  : boolean := true;
8          ENABLE_CRC          : boolean := true;
9      );
10     port (
11         clk_i      : in  std_ulogic;
12         rstn_i     : in  std_ulogic;
13         enable_i    : in  std_ulogic;
14         valid_o     : out std_ulogic;
15         data_o      : out std_ulogic_vector(7 downto 0)
16     );
17 end entity;

```

RYSUNEK 3.2: Parametry generic oraz porty neoTRNG.

3.1.1 Modyfikacje wprowadzone w module neoTRNG

Dodatkowo zaimplementowane zostały dwa kolejne parametry generyczne w celu zbadania wpływu obecności ekstraktora von Neumanna oraz wielomianu CRC:

- ENABLE_VON_NEUMANN – określa, czy sygnał ze źródła zostanie przetworzony przez ekstraktor von Neumanna. Gdy parametr jest równy True, sygnał jest kierowany do ekstraktora, w przeciwnym razie jest przekierowywany do następnego modułu,

- **ENABLE_CRC** – określa, czy sygnał z poprzedniego modułu zostanie przetworzony za pomocą wielomianu CRC. Gdy parametr jest równy **True**, sygnał jest kierowany do CRC, w przeciwnym razie moduł CRC jest omijany.

Pozwala to na zbadanie sygnału na różnych etapach pracy generatora.

TABELA 3.1: Wpływ konfiguracji parametrów generycznych na rzeczywisty sygnał wyjściowy generatora

ENABLE_VON_NEUMANN	ENABLE_CRC	Charakterystyka sygnału wyjściowego
False	False	Sygnał surowy (bezpośrednio ze źródła). Brak korekcji asymetrii i dodatkowego rozproszenia statystycznego.
False	True	Sygnał surowy przetworzony wyłącznie przez wielomian CRC. Poprawione właściwości statystyczne, lecz ewentualna asymetria źródła może wpływać na wynik końcowy.
True	False	Sygnał poddany ekstrakcji von Neumanna z pominięciem modułu CRC. Usunięta asymetria kosztem zmniejszenia przepustowości strumienia danych.
True	True	Pełna implementacja sprzętowa. Sygnał po ekstrakcji von Neumanna zostaje dodatkowo przetworzony przez wielomian CRC. Sygnał o możliwie najlepszych właściwościach statystycznych kosztem największego zużycia zasobów oraz najmniejszej przepustowości.

3.1.2 Opis pracy komórki entropii neoTRNG_cell

Bazowym modułem generatora neoTRNG jest neoTRNG_cell, który stanowi pojedynczy RO. Rozwiązanie to zaimplementowano za pomocą instrukcji generowania oraz dwóch procesów:

```

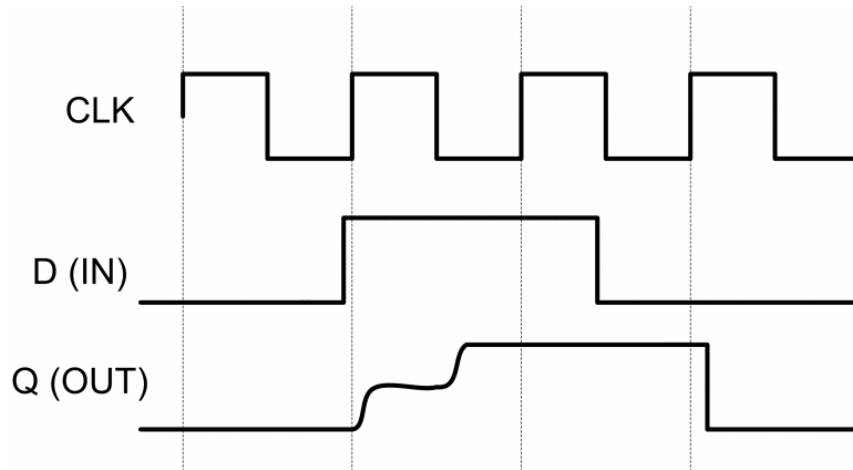
1  entity neoTRNG_cell is
2    generic (
3      NUM_INV   : natural;
4      SIM_MODE  : boolean
5    );
6    port (
7      clk_i     : in  std_ulogic;
8      rstn_i    : in  std_ulogic;
9      en_i      : in  std_ulogic;
10     en_o      : out std_ulogic;
11     rnd_o     : out std_ulogic
12   );
13 end entity;
```

RYSUNEK 3.3: Parametry generic oraz porty neoTRNG_cell.

1. **ring_osc_gen** – instrukcja generowania, która tworzy pętlę inwerterów oraz logikę zatrzaśków (ang. *latch*) pomiędzy nimi,

2. **en_shift_reg** – proces obsługujący rejestr przesuwany z sygnałami **enable** dla kolejnych zatrząsków w RO,
3. **synchronizer** – proces obsługujący dwustopniowy rejestr przesuwany (bufor próbek przed wyjściem).

Zatrząski pomiędzy inwerterami pozwalają na krokowe łączenie inwerterów z każdym cyklem zegara, dzięki czemu urządzenie włączane jest w powtarzalny sposób. Z kolei gdy jest wyłączone, cały układ pozostaje rozłączony. Funkcjonalność ta ma na celu zapobieżenie powstaniu metastabilności w układzie.



RYSUNEK 3.4: Metastabilność – stan nieustalony [10].

Metastabilność (ang. *metastability*) to zjawisko w elektronice cyfrowej, w którym przerzutnik nie potrafi w założonym czasie osiągnąć stanu ustalonego '0' lub '1'. Pozostaje wówczas przez pewien moment w stanie nieustalonym, co może prowadzić do błędów w innych częściach układu. Przyczyną metastabilności może być próbkowanie sygnału w trakcie zmiany stanu.

Ostateczną metodą uniknięcia metastabilności w układzie jest dwustopniowy rejestr przesuwany procesu synchronizatora. Jeżeli próbkowane jest wyjście z ostatniego RO i akurat wtedy sygnał znajduje się w stanie nieustalonym, oznacza to propagację wadliwego sygnału do reszty układu. Dodatkowy rejestr spowalnia proces generowania pierwszej próbki o jeden cykl, ale jego obecność w układzie zapobiega propagacji powyższego błędu. Wadliwy sygnał w pierwszym rejestrze ma czas na to, aby się ustalić.

Ostatecznie sygnał z drugiego rejestru jest przesuwany na wyjście układu **rnd_o** oraz podnoszony jest stan wysoki na linii **en_o**.

3.2 Projekt i implementacja macierzowego ekstraktora algebraicznego Toeplitza

Kolejna implementacja generatora zastępuje ekstraktor von Neumanna i wielomian CRC ekstraktorem Toeplitza. Wykorzystana została otwartoźródłowa implementacja dostępna w repozytorium (<https://github.com/rokzitzko/toeplitz>). Główne cechy charakterystyczne zastosowanego ekstraktora [24]:

- dostępne tryby pracy szeregowej i równoległej,

- modyfikowalna szerokość bloku wejściowego i wyjściowego,
- stałe ziarno macierzy zaszyte wewnątrz pliku RTL,
- dynamiczne generowanie kolumn macierzy Toeplitza, co pozwala uniknąć przechowywania całej macierzy naraz w układzie.

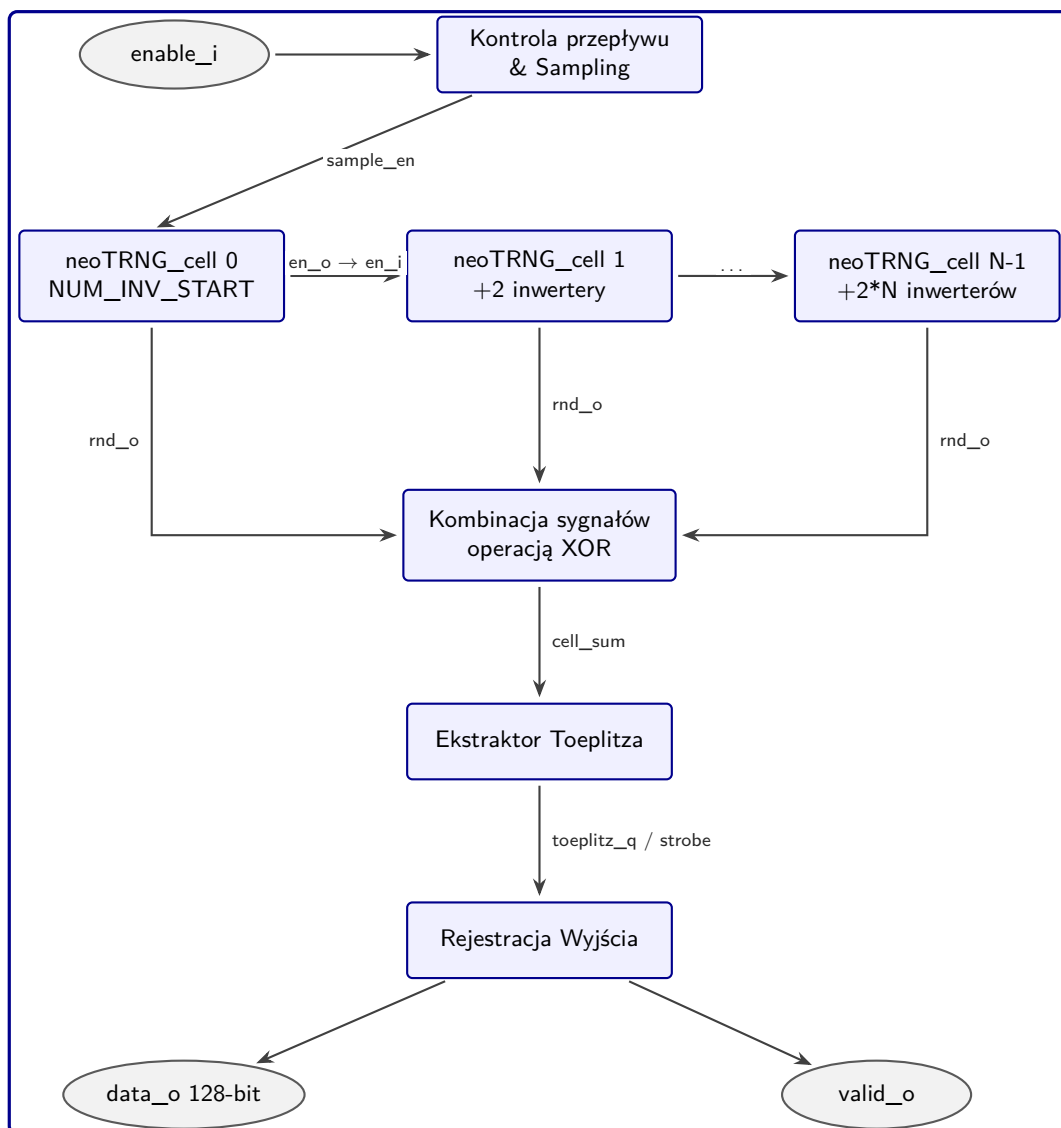
Integracja generatora neoTRNG z nowym modulem wymagała wstępnego przetłumaczenia opisu ekstraktora z języka Verilog na VHDL. Poprawność wykonanego tłumaczenia została zweryfikowana za pomocą środowiska testowego (ang. *testbench*) zaimplementowanego w narzędzi Vivado. W ramach przeprowadzonej procedury testowej dla ekstraktora Toeplitza, dla zadanych danych wejściowych generowane były wartości wyjściowe, które następnie automatycznie porównywano z wartościami oczekiwanymi, co potwierdziło poprawność działania modułu.

```
1 Note: Extracted Value (q): 0x6743ABD32AF0540CCEAF400958CD8550
2 Note: Expected Value:      0x6743ABD32AF0540CCEAF400958CD8550
3 Note: VERIFICATION RESULT: PASS
```

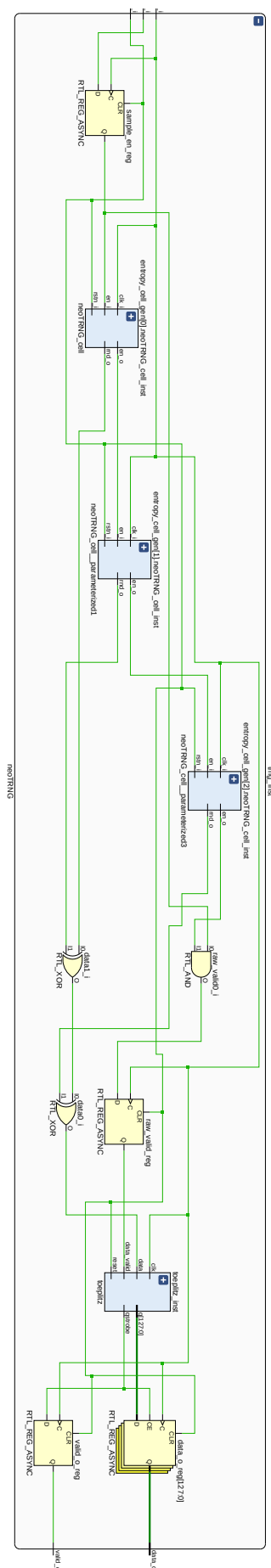
RYSUNEK 3.5: Wynik symulacji ekstraktora Toeplitza.

Wynik z symulacji potwierdza poprawne przetłumaczenie modułu. Na poniższych schematach (Fig. 3.6 oraz Fig. 3.7) widać położenie ekstraktora Toeplitza tuż za komórkami entropii. Został on wbudowany wewnątrz modułu neoTRNG w miejsce logiki ekstraktora von Neumanna i wielomianu CRC.

System działa w trybie szeregowym. Generator neoTRNG zwraca kolejne bity, które są kolejowane do ekstraktora. Wynik jest mnożony przez macierz Toeplitza oraz wyprowadzany na linii `data_o` do nadrzędnego modułu komunikacji, o czym sygnalizuje linia `valid_o`.



RYSUNEK 3.6: Diagram ideowy modułu neoTRNG zintegrowanego z ekstraktorem Toeplitza.



RYSUNEK 3.7: Schemat generatora z ekstraktorem Toeplitza.

3.3 Projekt i implementacja ekstraktora kryptograficznego AES-CTR

Ostatnia implementacja wprowadza postprocessing z wykorzystaniem szyfru strumieniowego AES-CTR. Blok AES-CTR został opracowany na podstawie otwartoźródłowej implementacji silnika szyfru blokowego AES, dostępnej w repozytorium (<https://opencores.org/projects/aes>).

Główne cechy zastosowanej implementacji silnika AES [6]:

- realizuje zarówno szyfrowanie, jak i deszyfrowanie AES dla klucza o długości 128 bitów,
- architektura typu loop-unrolled (round-based) – implementacja realizuje jedną rundę AES na cykl zegara i wykorzystuje ją wielokrotnie, co zmniejsza zużycie zasobów FPGA,
- obliczanie nowego szyfrogramu co 10 cykli zegara,
- implementacja w pełni przetestowana i zweryfikowana za pomocą wektorów testowych NIST [2].

Projekt silnika udostępnia również środowisko testowe, który automatycznie porównuje otrzymane wyniki operacji szyfrowania z wartościami referencyjnymi, w celu potwierdzenia poprawności pracy szyfru.

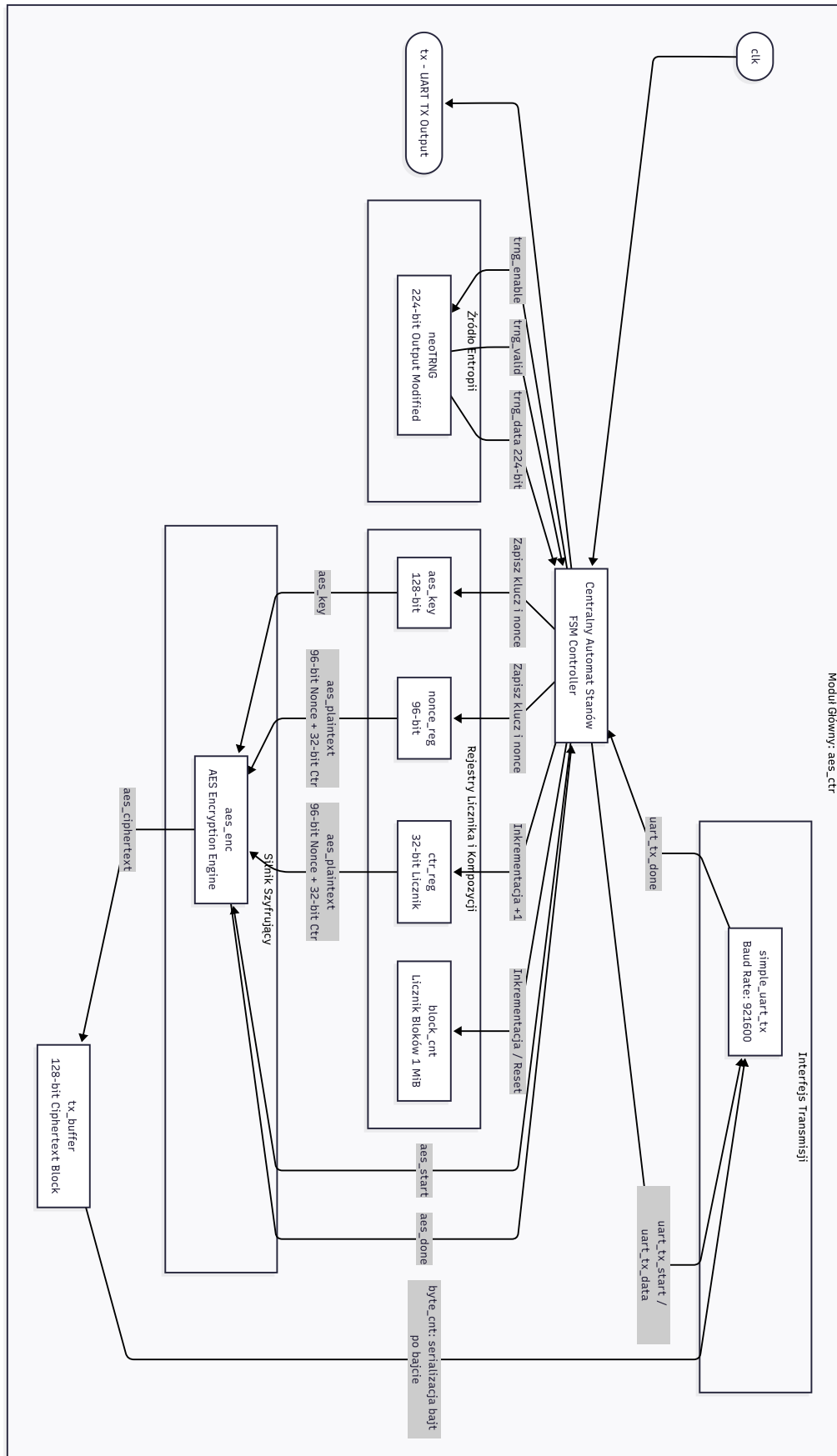
```

1 Note: ----- Passed -----
2 Note: Expected: 320b6a19978511dcfb09dc021d842539
3 Note: Received: 320B6A19978511DCFB09DC021D842539
4 Note: ----- Passed -----
5 Note: Expected: 2e2b34ca59fa4c883b2c8aefd44be966
6 Note: Received: 2E2B34CA59FA4C883B2C8AEFD44BE966
7 Note: ----- Passed -----
8 Note: Expected: 97ef6624f3ca9ea860367a0db47bd73a
9 Note: Received: 97EF6624F3CA9EA860367A0DB47BD73A

```

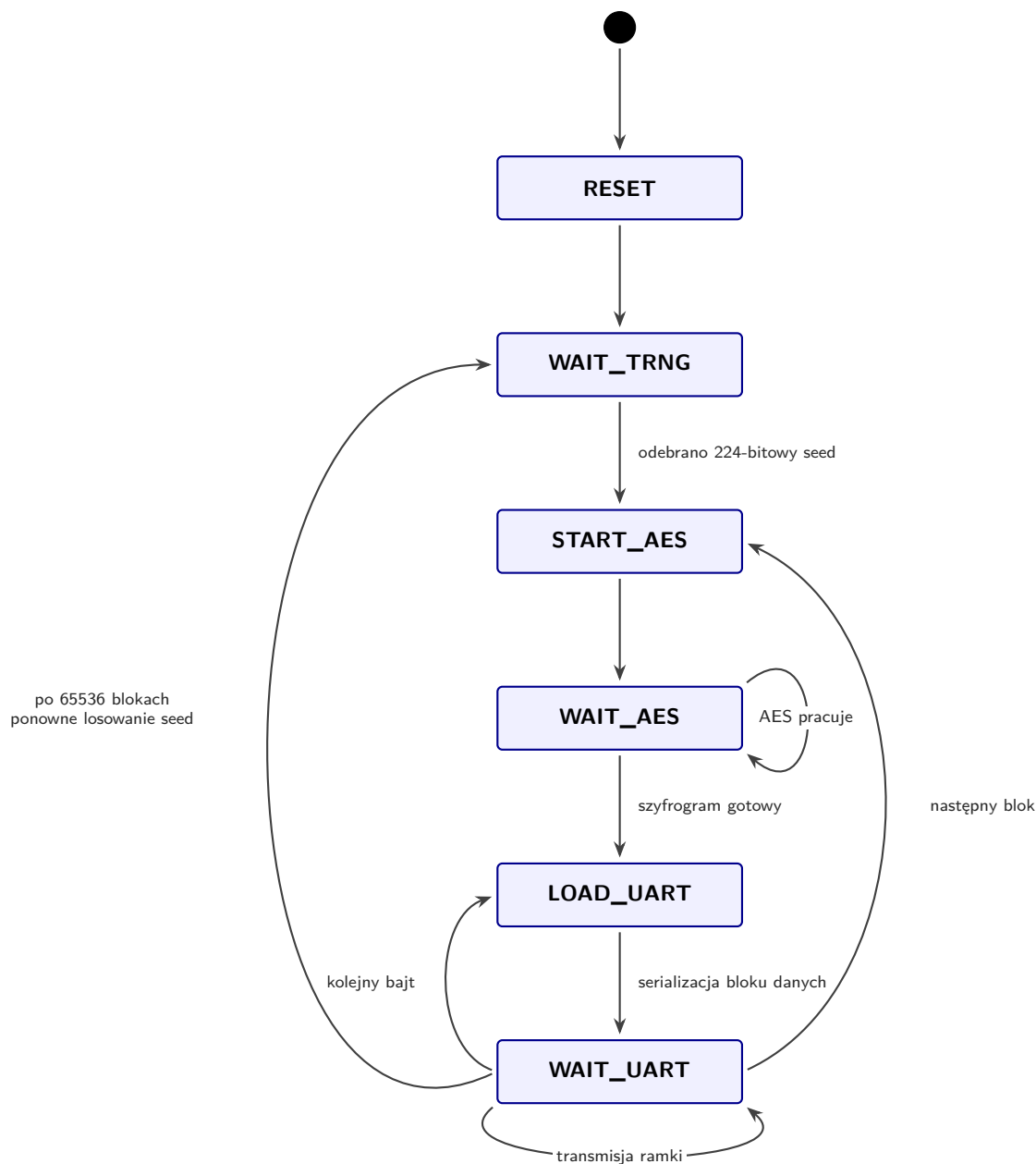
RYSUNEK 3.8: Wyniki symulacji silnika AES.

Poniższy diagram blokowy (Fig. 3.9) przedstawia przepływ sygnałów w komponencie AES-CTR. Sercem układu jest centralna maszyna stanów, która zarządza współpracą generatora, rejestrów, silnika AES i modułu komunikacji. Proces rozpoczyna się w źródle entropii, które dostarcza 224 bity losowe dla klucza w `aes_key` i nonce w `nonce_reg`. Równocześnie nadzorowana jest praca 32-bitowego licznika, którego zawartość kopiowana jest wraz z nonce do `aes_plaintext`. W wyniku pracy silnika AES powstaje 128-bitowy blok danych, który trafia do modułu komunikacji i jest przesyłany jako szesnaście 8-bitowych ramek. Po zakończeniu transmisji licznik inkrementuje swoją wartość, a silnik generuje nowy blok dla modułu transmisji. Licznik inkrementowany jest 65536 razy, co łącznie daje 1 MB danych losowych wygenerowanych z jednego ziarna pobranego z neoTRNG. Następnie pobierane jest nowe ziarno i cały proces się powtarza.



RYSUNEK 3.9: Architektura modułu nadrzędnego AES-CTR wyposażonego w maszynę stanów, generator neoTRNG, silnik AES, rejestr licznika oraz moduł komunikacji.

Praca maszyny stanów opisana jest za pomocą siedmiu stanów widocznych na schemacie (Fig. 3.10):



RYSUNEK 3.10: Diagram stanów kontrolera pracy generatora z silnikiem AES-CTR.

3.4 Parzysta liczba inwerterów

Poważnym błędem konfiguracyjnym generatora opartego o RO jest ustawienie parzystej liczby inwerterów wewnątrz RO. Nieparzysta liczba wymusza oscylacje wewnątrz układu, w przeciwnym razie sygnał inwerterów się ustala oraz powinien zwracać stały sygnał '1' lub '0'. Stanowi to potencjalnie nowy wektor ataku, gdzie poprzez modyfikację parametru `NUM_INV_START` możliwe jest wymuszenie paraliżu całego generatora. W rezultacie moduły kryptograficzne pobierające entropię z wadliwego źródła przestaną być bezpieczne.

```

1  trng_inst : entity work.neoTRNG
2      generic map (
3          NUM_CELLS          => 2,
4          NUM_INV_START      => 2,
5          NUM_RAW_BITS       => 64,
6          SIM_MODE           => false,
7          ENABLE_VON_NEUMANN => false,
8          ENABLE_CRC         => false
9      )

```

RYSUNEK 3.11: Generator z parzystą liczbą RO.

Aby możliwe było wykorzystanie parzystej liczby RO w planowaniu ataku, należy zapewnić dezaktywację ekstraktora Von Neumanna. Jeżeli pozostanie on aktywny, wówczas wartości pobierane ze źródła entropii zostaną odrzucone i cały proces generacji liczby losowej zostanie zatrzymany. Przykłady takiego wektora ataków są widoczne na Fig. 3.12 i Fig. 3.13.

```

1  $ sudo xxd /dev/ttyUSB0
2  00000000: ffff ffff ffff ffff ffff ffff ffff ffff .....
3  00000010: ffff ffff ffff ffff ffff ffff ffff ffff .....
4  00000020: ffff ffff ffff ffff ffff ffff ffff ffff .....
5  00000030: ffff ffff ffff ffff ffff ffff ffff ffff .....

```

RYSUNEK 3.12: Ciągły sygnał '1' z generatora dla parzystej liczby inwerterów.

```

1  $ sudo xxd /dev/ttyUSB0
2  00000000: 0000 0000 0000 0000 0000 0000 0000 0000 .....
3  00000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
4  00000020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
5  00000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....

```

RYSUNEK 3.13: Ciągły sygnał '0' z generatora dla parzystej liczby inwerterów.

3.5 Wykorzystanie zasobów sprzętowych matrycy FPGA

Dla każdej badanej implementacji oraz konfiguracji zostały zebrane raporty o zajętości sprzętowej układu. Poniżej znajduje się całe zestawienie (Tab. 3.2)

TABELA 3.2: Compiled Resource Utilization Results across Chapters

post. proc.	RO	I	VN	CRC	LUTs	Registers	F7 Mux	F8 Mux
VN-CRC	2	8I	False	False	39	39	0	0
VN-CRC	2	8I	False	True	37	41	0	0
VN-CRC	2	8I	True	False	38	42	0	0
VN-CRC	2	8I	True	True	44	44	0	0
VN-CRC	3	15I	False	False	60	41	0	0
VN-CRC	3	15I	False	True	61	43	0	0
VN-CRC	3	15I	True	False	52	44	0	0
VN-CRC	3	15I	True	True	50	46	0	0
VN-CRC	3	21I	False	False	61	41	0	0
VN-CRC	3	21I	False	True	72	43	0	0
VN-CRC	3	21I	True	False	65	44	0	0
VN-CRC	3	21I	True	True	69	46	0	0
VN-CRC	2 D	8I	False	False	46	39	0	0
VN-CRC	2 D	8I	False	True	42	41	0	0
VN-CRC	2 D	8I	True	False	47	42	0	0
VN-CRC	2 D	8I	True	True	43	44	0	0
VN-CRC	2 L	8I	False	False	37	39	0	0
VN-CRC	2 L	8I	False	True	40	41	0	0
VN-CRC	2 L	8I	True	False	38	42	0	0
VN-CRC	2 L	8I	True	True	42	44	0	0
VN-CRC	3 D	21I	False	False	62	41	0	0
VN-CRC	3 D	21I	False	True	77	43	0	0
VN-CRC	3 D	21I	True	False	66	44	0	0
VN-CRC	3 D	21I	True	True	69	46	0	0
VN-CRC	3 L	21I	False	False	67	41	0	0
VN-CRC	3 L	21I	False	True	79	43	0	0
VN-CRC	3 L	21I	True	False	62	44	0	0
VN-CRC	3 L	21I	True	True	70	46	0	0
VN-CRC	6 D	60I	False	False	150	47	0	0
VN-CRC	6 D	60I	False	True	162	49	0	0
VN-CRC	6 D	60I	True	False	153	50	0	0
VN-CRC	6 D	60I	True	True	162	52	0	0
VN-CRC	6 L	60I	False	False	142	47	0	0
VN-CRC	6 L	60I	False	True	145	49	0	0
VN-CRC	6 L	60I	True	False	167	50	0	0
VN-CRC	6 L	60I	True	True	169	52	0	0
AES-CTR	2	8I	N/A	N/A	1276	1317	272	136
AES-CTR	3	15I	N/A	N/A	1284	1333	272	136
Toeplitz	2	8I	N/A	N/A	261	926	0	0
Toeplitz	3	15I	N/A	N/A	263	928	0	0

Legenda:

post. proc. – zastosowana forma przetwarzania końcowego.

RO – liczba oscylatorów pierścieniowych w układzie.

D – rozproszone rozmieszczenie oscylatorów.

L – zlokalizowane rozmieszczenie oscylatorów.

I – suma inwerterów we wszystkich oscylatorach łącznie.

VN – obecność (True) bądź brak (False) ekstraktora von Neumanna.

CRC – obecność (True) bądź brak (False) wielomianu CRC.

LUTs – liczba wykorzystanych tablic look-up.

Registers – liczba wykorzystanych rejestrów przerzutnikowych.

F7 Mux – liczba wykorzystanych multiplexerów strukturalnych F7.

F8 Mux – liczba wykorzystanych multiplexerów strukturalnych F8.

Rozdział 4

Badania eksperymentalne

4.1 Wpływ obecności post-processingu oraz liczby oscylatorów RO na losowość

Pierwsza seria badań skupia się na bazowej konfiguracji neoTRNG z ekstraktorem von Neumanna i wielomianem CRC. Zbadane zostały następujące konfiguracje:

- 2 RO po 3 i 5 inwerterów,
- 3 RO po 3, 5 i 7 inwerterów,
- 3 RO po 5, 7 i 9 inwerterów.

Dla każdej z konfiguracji sprawdzono osobno wpływ obecności korektora von Neumanna i wielomianu CRC, co dało łącznie dwanaście różnych konfiguracji generatora.

4.1.1 NIST SP 800-22

Analiza losowości przy użyciu testów NIST SP 800-22 dla konfiguracji z 2RO 8I, 3RO 15I, 3RO 21I w neoTRNG. Testy przeprowadzono na płytce deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

Uwaga: Minimalny wskaźnik zaliczenia dla każdego testu statystycznego, z wyjątkiem testu błędzenia losowego (w wariancie *random excursion / random excursion variant*), wynosi w przybliżeniu 96 dla próby o wielkości 100 sekwencji binarnych [14].

Uwaga: Minimalny wskaźnik zaliczenia dla każdego testu statystycznego, z wyjątkiem testu błędzenia losowego (w wariancie *random excursion / random excursion variant*), wynosi w przybliżeniu 96 dla próby o wielkości 100 sekwencji binarnych [14].

Legenda: sp – liczba zaliczonych podtestów (ang. *subtests passed*).

Legenda: * – test niezaliczony.

Legenda: *ENABLE_VON_NEUMANN* / *ENABLE_CRC* – wybrana konfiguracja.

TABELA 4.1: NIST 800-22 : 2 RO : 3-5

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	100/100		100/100		98/100		100/100	
BlockFrequency	100/100		100/100		98/100		100/100	
CumulativeSums	100/100		100/100		98/100		100/100	
CumulativeSums	100/100		100/100		98/100		100/100	
Runs	100/100		0/100	*	99/100		0/100	*
LongestRun	98/100		0/100	*	99/100		0/100	*
Rank	99/100		98/100		99/100		98/100	
FFT	100/100		0/100	*	98/100		0/100	*
NonOverlappingTemplate	148/148	sp	2/148	sp	148/148	sp	0/148	sp
OverlappingTemplate	99/100		0/100	*	100/100		0/100	*
Universal	98/100		0/100	*	100/100		0/100	*
ApproximateEntropy	98/100		0/100	*	97/100		0/100	*
RandomExcursions	8/8	sp	0/6	sp	8/8	sp	0/6	sp
RandomExcursionsVariant	18/18	sp	4/18	sp	18/18	sp	0/18	sp
Serial	99/100		0/100	*	98/100		0/100	*
Serial	100/100		0/100	*	99/100		0/100	*
LinearComplexity	98/100		98/100		100/100		100/100	

TABELA 4.2: NIST 800-22 : 3 RO : 3-5-7

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	98/100		100/100		98/100		100/100	
BlockFrequency	100/100		100/100		97/100		100/100	
CumulativeSums	98/100		99/100		99/100		100/100	
CumulativeSums	99/100		100/100		98/100		100/100	
Runs	100/100		0/100	*	98/100		35/100	*
LongestRun	99/100		0/100	*	99/100		25/100	*
Rank	99/100		100/100		100/100		100/100	
FFT	98/100		75/100	*	99/100		0/100	*
NonOverlappingTemplate	147/148	sp	36/148	sp	148/148	sp	0/148	sp
OverlappingTemplate	100/100		0/100	*	99/100		0/100	*
Universal	98/100		68/100	*	96/100		0/100	*
ApproximateEntropy	100/100		0/100	*	99/100		0/100	*
RandomExcursions	8/8	sp	8/8	sp	8/8	sp	2/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	99/100		0/100	*	98/100		0/100	*
Serial	99/100		99/100		99/100		0/100	*
LinearComplexity	97/100		100/100		100/100		99/100	

TABELA 4.3: NIST 800-22 : 3 RO : 5-7-9

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	98/100		100/100		97/100		99/100	
BlockFrequency	99/100		100/100		98/100		100/100	
CumulativeSums	98/100		100/100		97/100		100/100	
CumulativeSums	98/100		100/100		97/100		100/100	
Runs	100/100		32/100	*	97/100		0/100	*
LongestRun	100/100		0/100	*	99/100		20/100	*
Rank	99/100		99/100		99/100		99/100	
FFT	100/100		0/100	*	98/100		29/100	*
NonOverlappingTemplate	147/148	sp	10/148	sp	148/148	sp	30/148	sp
OverlappingTemplate	100/100		0/100	*	99/100		1/100	*
Universal	97/100		0/100	*	100/100		21/100	*
ApproximateEntropy	100/100		0/100	*	100/100		0/100	*
RandomExcursions	7/8	sp	1/8	sp	8/8	sp	8/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	17/18	sp
Serial	99/100		0/100	*	99/100		0/100	*
Serial	98/100		91/100	*	99/100		95/100	*
LinearComplexity	99/100		99/100		100/100		99/100	

4.1.2 NIST SP 800-90B

Analiza losowości przy użyciu testów NIST SP 800-90B dla konfiguracji z 2RO 8I, 3RO 15I, 3RO 21I w neoTRNG. Testy przeprowadzono na płytce deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

TABELA 4.4: NIST 800-90 : 3 RO 5-7-9 / 3 RO 3-5-7 / 2 RO 3-5

Konfiguracja	3RO 5-7-9	3RO 3-5-7	2RO 3-5
true/true	$H_{min} = 7.529456$	$H_{min} = 7.493927$	$H_{min} = 7.517294$
true/false	$H_{min} = 6.449056$	$H_{min} = 7.194429$	$H_{min} = 3.317881$
false/true	$H_{min} = 7.529456$	$H_{min} = 7.517294$	$H_{min} = 7.529456$
false/false	$H_{min} = 6.441963$	$H_{min} = 4.746528$	$H_{min} = 2.274360$

Legenda: $H_{min} = \text{Min}(H_{original}, 8X H_{bitstring})$

4.1.3 ENT

Analiza losowości przy użyciu testu ENT dla konfiguracji z 2RO 8I, 3RO 15I, 3RO 21I w neoTRNG. Testy przeprowadzono na płytce deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

TABELA 4.5: ENT : 3 RO 5-7-9 / 3 RO 3-5-7 / 2 RO 3-5

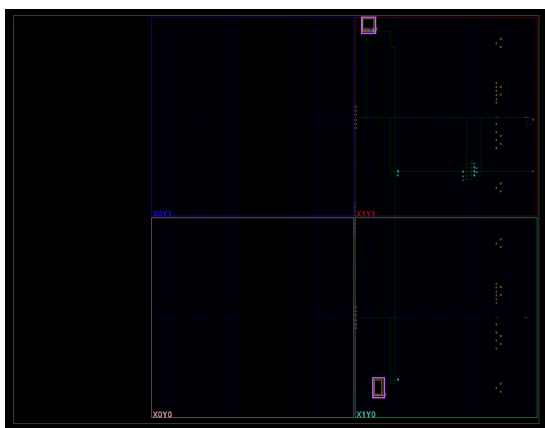
Konfiguracja	3RO 5-7-9	3RO 3-5-7	2RO 3-5
true/true	Entropy = 7.999998	Entropy = 7.999998	Entropy = 7.999997
true/false	Entropy = 7.864606	Entropy = 7.977855	Entropy = 7.168778
false/true	Entropy = 7.999988	Entropy = 7.999995	Entropy = 7.999742
false/false	Entropy = 7.917160	Entropy = 7.398469	Entropy = 5.322542

Legenda: Entropy = X bits per byte.

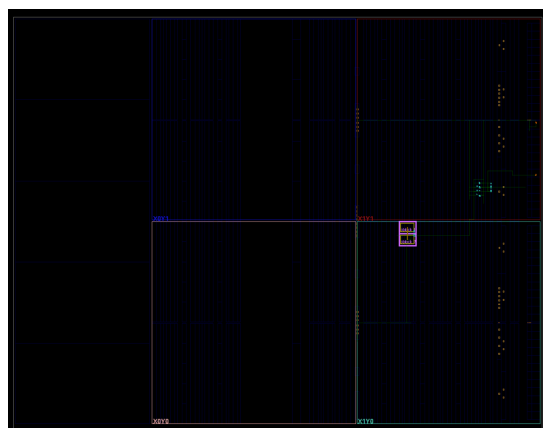
4.2 Wpływ rozmieszczenia RO w układzie FPGA na losowość

Narzędzie Vivado umożliwia ręczne planowanie rozmieszczenia RO wewnątrz układu FPGA. Zostało to opisane w załączniku A. Poniżej przedstawiono widoki z narzędzia do planowania roz-

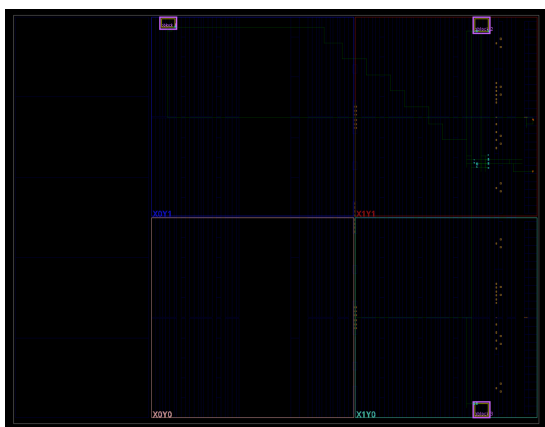
mieszczenia dla każdej z konfiguracji:



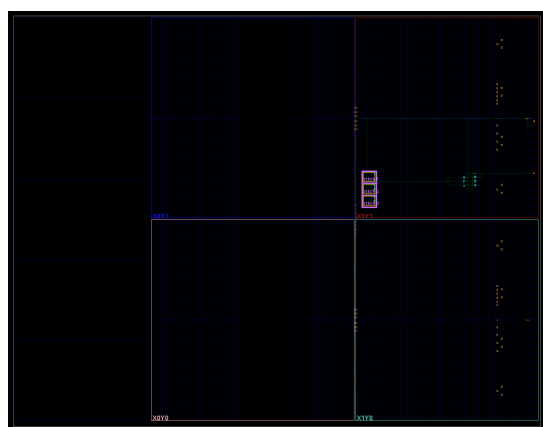
RYSUNEK 4.1: 2 RO rozproszone.



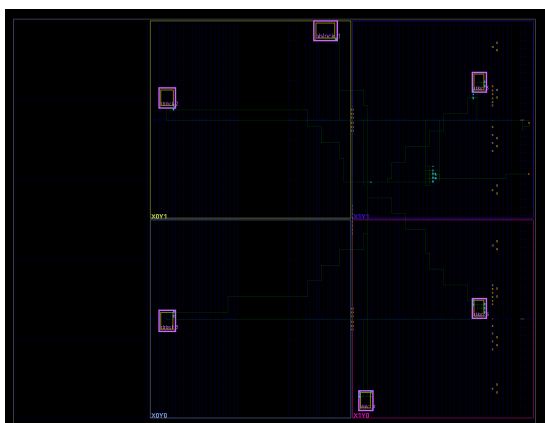
RYSUNEK 4.2: 2 RO zlokalizowane.



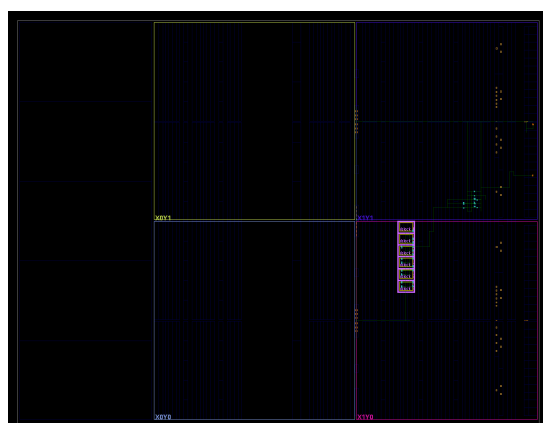
RYSUNEK 4.3: 3 RO rozproszone.



RYSUNEK 4.4: 3 RO zlokalizowane.



RYSUNEK 4.5: 6 RO rozproszone.



RYSUNEK 4.6: 6 RO zlokalizowane.

Druga seria badań skupia się na wpływie rozmieszczenia RO wewnątrz układu FPGA bazowego

generatora neoTRNG z ekstraktorem von Neumanna i wielomianem CRC. Zbadane zostały następujące konfiguracje:

- 2 RO po 3 i 5 inwerterów – RO rozproszone,
- 2 RO po 3 i 5 inwerterów – RO zlokalizowane,
- 3 RO po 5, 7 i 9 inwerterów – RO rozproszone,
- 3 RO po 5, 7 i 9 inwerterów – RO zlokalizowane,
- 6 RO po 5, 7, 9, 11, 13, 15 inwerterów – RO rozproszone,
- 6 RO po 5, 7, 9, 11, 13, 15 inwerterów – RO zlokalizowane.

Dla każdej z konfiguracji sprawdzono osobno wpływ obecności korektora von Neumanna i wielomianu CRC, co dało łącznie 24 różne konfiguracje generatora.

4.2.1 NIST SP 800-22

Analiza losowości przy użyciu testów NIST SP 800-22 dla konfiguracji z 2RO 8I rozmieszczenie rozproszone, 2RO 8I rozmieszczenie zlokalizowane, 3RO 21I rozmieszczenie rozproszone, 3RO 21I rozmieszczenie zlokalizowane, 6RO 60I rozmieszczenie rozproszone, 6RO 60I rozmieszczenie zlokalizowane w neoTRNG. Testy przeprowadzono na płycie deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

Uwaga: Minimalny wskaźnik zaliczenia dla każdego testu statystycznego, z wyjątkiem testu błędzenia losowego (w wariancie *random excursion / random excursion variant*), wynosi w przybliżeniu 96 dla próby o wielkości 100 sekwencji binarnych [14].

Uwaga: Minimalny wskaźnik zaliczenia dla każdego testu statystycznego, z wyjątkiem testu błędzenia losowego (w wariancie *random excursion / random excursion variant*), wynosi w przybliżeniu 96 dla próby o wielkości 100 sekwencji binarnych [14].

Legenda: sp – liczba zaliczonych podtestów (ang. *subtests passed*).

Legenda: * – test niezaliczony.

Legenda: *ENABLE_VON_NEUMANN / ENABLE_CRC* – wybrana konfiguracja.

TABELA 4.6: NIST 800-22 : 2 RO : 3-5 rozproszone

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	99/100		100/100		97/100		100/100	
BlockFrequency	96/100		100/100		95/100	*	100/100	
CumulativeSums	99/100		100/100		97/100		100/100	
CumulativeSums	98/100		100/100		97/100		100/100	
Runs	100/100		0/100	*	86/100	*	0/100	*
LongestRun	100/100		0/100	*	100/100		0/100	*
Rank	100/100		99/100		100/100		98/100	
FFT	100/100		0/100	*	99/100		0/100	*
NonOverlappingTemplate	148/148	sp	5/148	sp	144/148	sp	0/148	sp
OverlappingTemplate	98/100		0/100	*	96/100		0/100	*
Universal	98/100		0/100	*	99/100		0/100	*
ApproximateEntropy	98/100		0/100	*	87/100	*	0/100	*
RandomExcursions	8/8	sp	0/0	sp	8/8	sp	0/8	sp
RandomExcursionsVariant	18/18	sp	0/18	sp	18/18	sp	18/18	sp
Serial	100/100		0/100	*	98/100		0/100	*
Serial	100/100		0/100	*	97/100		0/100	*
LinearComplexity	100/100		100/100		98/100		100/100	

TABELA 4.7: NIST 800-22 : 2 RO : 3-5 zlokalizowane

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	99/100		100/100		98/100		19/100	*
BlockFrequency	100/100		100/100		99/100		100/100	
CumulativeSums	99/100		100/100		99/100		26/100	*
CumulativeSums	99/100		100/100		96/100		23/100	*
Runs	98/100		0/100	*	71/100	*	0/100	*
LongestRun	99/100		0/100	*	100/100		0/100	*
Rank	98/100		99/100		100/100		98/100	
FFT	98/100		0/100	*	99/100		0/100	*
NonOverlappingTemplate	148/148	sp	4/148	sp	111/148	sp	5/148	sp
OverlappingTemplate	99/100		0/100	*	89/100	*	0/100	*
Universal	96/100		0/100	*	98/100		0/100	*
ApproximateEntropy	98/100		0/100	*	9/100	*	0/100	*
RandomExcursions	8/8	sp	0/8	sp	8/8	sp	1/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	17/18	sp
Serial	100/100		0/100	*	96/100		0/100	*
Serial	99/100		11/100	*	100/100		0/100	*
LinearComplexity	100/100		97/100		98/100		100/100	

TABELA 4.8: NIST 800-22 : 3 RO : 5-7-9 rozproszone

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	100/100		99/100		96/100		96/100	
BlockFrequency	100/100		100/100		99/100		84/100	*
CumulativeSums	100/100		99/100		97/100		98/100	
CumulativeSums	98/100		99/100		97/100		97/100	
Runs	98/100		0/100	*	100/100		99/100	
LongestRun	100/100		46/100	*	100/100		29/100	*
Rank	99/100		99/100		98/100		99/100	
FFT	98/100		45/100	*	99/100		43/100	*
NonOverlappingTemplate	146/148	sp	19/148	sp	148/148	sp	0/148	sp
OverlappingTemplate	98/100		21/100	*	99/100		2/100	*
Universal	97/100		18/100	*	99/100		8/100	*
ApproximateEntropy	98/100		0/100	*	99/100		0/100	*
RandomExcursions	8/8	sp	5/8	sp	8/8	sp	6/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	100/100		0/100	*	98/100		0/100	*
Serial	99/100		96/100		97/100		100/100	
LinearComplexity	99/100		100/100		99/100		99/100	

TABELA 4.9: NIST 800-22 : 3 RO : 5-7-9 zlokalizowane

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	99/100		100/100		98/100		100/100	
BlockFrequency	100/100		95/100	*	97/100		39/100	*
CumulativeSums	99/100		100/100		98/100		100/100	
CumulativeSums	99/100		100/100		98/100		100/100	
Runs	97/100		0/100	*	99/100		0/100	*
LongestRun	98/100		55/100	*	99/100		68/100	*
Rank	100/100		99/100		100/100		98/100	
FFT	100/100		32/100	*	100/100		0/100	*
NonOverlappingTemplate	147/148	sp	19/148	sp	148/148	sp	2/148	sp
OverlappingTemplate	98/100		50/100	*	99/100		0/100	*
Universal	100/100		56/100	*	98/100		0/100	*
ApproximateEntropy	100/100		0/100	*	95/100	*	0/100	*
RandomExcursions	8/8	sp	7/8	sp	8/8	sp	0/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	98/100		0/100	*	97/100		0/100	*
Serial	99/100		96/100		99/100		0/100	*
LinearComplexity	100/100		98/100		100/100		100/100	

TABELA 4.10: NIST 800-22 : 6 RO : 5-7-9-11-13-15 rozproszone

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	100/100		99/100		98/100		99/100	
BlockFrequency	98/100		100/100		100/100		0/100	*
CumulativeSums	99/100		99/100		98/100		99/100	
CumulativeSums	100/100		99/100		98/100		97/100	
Runs	99/100		61/100	*	97/100		99/100	
LongestRun	100/100		60/100	*	98/100		41/100	*
Rank	99/100		100/100		100/100		99/100	
FFT	96/100		98/100		99/100		43/100	*
NonOverlappingTemplate	148/148	sp	109/148	sp	147/148	sp	0/148	sp
OverlappingTemplate	99/100		2/100	*	97/100		0/100	*
Universal	99/100		100/100		99/100		32/100	*
ApproximateEntropy	100/100		2/100	*	99/100		0/100	*
RandomExcursions	8/8	sp	8/8	sp	8/8	sp	8/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	99/100		85/100	*	100/100		0/100	*
Serial	100/100		98/100		100/100		98/100	
LinearComplexity	100/100		100/100		99/100		99/100	

TABELA 4.11: NIST 800-22 : 6 RO : 5-7-9-11-13-15 zlokalizowane

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	99/100		98/100		100/100		100/100	
BlockFrequency	100/100		99/100		99/100		63/100	*
CumulativeSums	98/100		98/100		99/100		100/100	
CumulativeSums	99/100		97/100		100/100		100/100	
Runs	100/100		0/100	*	96/100		100/100	
LongestRun	100/100		98/100		100/100		97/100	
Rank	99/100		98/100		100/100		99/100	
FFT	100/100		100/100		99/100		99/100	
NonOverlappingTemplate	148/148	sp	102/148	sp	148/148	sp	118/148	sp
OverlappingTemplate	100/100		99/100		97/100		75/100	*
Universal	100/100		95/100	*	98/100		100/100	
ApproximateEntropy	99/100		14/100	*	99/100		22/100	*
RandomExcursions	8/8	sp	8/8	sp	8/8	sp	8/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	100/100		96/100		100/100		94/100	*
Serial	99/100		98/100		100/100		98/100	
LinearComplexity	100/100		97/100		98/100		97/100	

4.2.2 NIST SP 800-90B

Analiza losowości przy użyciu testów NIST SP 800-90B dla konfiguracji z 2RO 8I rozmieszczenie rozproszone, 2RO 8I rozmieszczenie zlokalizowane, 3RO 21I rozmieszczenie rozproszone, 3RO 21I rozmieszczenie zlokalizowane, 6RO 60I rozmieszczenie rozproszone, 6RO 60I rozmieszczenie zlokalizowane w neoTRNG. Testy przeprowadzono na płycie deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

TABELA 4.12: NIST 800-90 : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 rozproszone

Konfiguracja	6RO 5-7-9-11-13-15 D	3RO 5-7-9 D	2RO 3-5 D
true/true	$H_{min} = 7.529456$	$H_{min} = 7.464863$	$H_{min} = 7.529456$
true/false	$H_{min} = 7.505457$	$H_{min} = 6.252241$	$H_{min} = 2.672020$
false/true	$H_{min} = 7.464863$	$H_{min} = 7.517294$	$H_{min} = 7.505457$
false/false	$H_{min} = 6.755574$	$H_{min} = 6.462812$	$H_{min} = 2.779685$

Legenda: $H_{min} = \text{Min}(H_{original}, 8XH_{bitstring})$

TABELA 4.13: NIST 800-90 : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 zlokalizowane

Konfiguracja	6RO 5-7-9-11-13-15 L	3RO 5-7-9 L	2RO 3-5 L
true/true	$H_{min} = 7.529456$	$H_{min} = 7.505457$	$H_{min} = 7.517294$
true/false	$H_{min} = 7.493927$	$H_{min} = 5.815950$	$H_{min} = 4.967096$
false/true	$H_{min} = 7.512159$	$H_{min} = 7.541960$	$H_{min} = 7.340455$
false/false	$H_{min} = 7.517294$	$H_{min} = 5.186556$	$H_{min} = 4.817879$

Legenda: $H_{min} = \text{Min}(H_{original}, 8XH_{bitstring})$

4.2.3 ENT

Analiza losowości przy użyciu testu ENT dla konfiguracji z 2RO 8I rozmieszczenie rozproszone, 2RO 8I rozmieszczenie zlokalizowane, 3RO 21I rozmieszczenie rozproszone, 3RO 21I rozmieszczenie zlokalizowane, 6RO 60I rozmieszczenie rozproszone, 6RO 60I rozmieszczenie zlokalizo-

wane w neoTRNG. Testy przeprowadzono na płycie deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

TABELA 4.14: ENT : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 rozproszone

Konfiguracja	6RO 5-7-9-11-13-15 D	3RO 3-5-7 D	2RO 3-5 D
true/true	Entropy = 7.999998	Entropy = 7.999998	Entropy = 7.999996
true/false	Entropy = 7.996747	Entropy = 7.960634	Entropy = 6.773816
false/true	Entropy = 7.999998	Entropy = 7.999923	Entropy = 7.999600
false/false	Entropy = 7.940526	Entropy = 7.956831	Entropy = 5.826810

Legenda: Entropy = X bits per byte.

TABELA 4.15: ENT : 6 RO 5-7-9-11-13-15 / 3 RO 5-7-9 / 3 RO 5-7-9 zlokalizowane

Konfiguracja	6RO 5-7-9-11-13-15 L	3RO 3-5-7 L	2RO 3-5 L
true/true	Entropy = 7.999998	Entropy = 7.999998	Entropy = 7.999998
true/false	Entropy = 7.998615	Entropy = 7.915655	Entropy = 7.771054
false/true	Entropy = 7.999998	Entropy = 7.999988	Entropy = 7.992428
false/false	Entropy = 7.998956	Entropy = 7.565826	Entropy = 7.308819

Legenda: Entropy = X bits per byte.

4.3 Wpływ ekstraktorów AES-CTR i Toeplitz na losowość przy zredukowanej liczbie RO

Ostatnia seria badań wprowadza modyfikację generatora w postaci nowych metod post-processingu. Wdrożone rozwiązania to ekstraktor Toeplitza oraz szyfr blokowy działający w trybie licznikowym (AES-CTR). Zbadane zostały następujące konfiguracje:

- 2 RO po 3 i 5 inwerterów oraz ekstraktor Toeplitza,
- 3 RO po 3, 5 i 7 inwerterów oraz ekstraktor Toeplitza,
- 2 RO po 3 i 5 inwerterów oraz AES-CTR,
- 3 RO po 3, 5 i 7 inwerterów oraz AES-CTR.

Powyższe podejście dało łącznie 4 różne konfiguracje generatora.

4.3.1 NIST SP 800-22

Analiza losowości przy użyciu testów NIST SP 800-22 dla konfiguracji z 2RO 8I z ekstraktorem Toeplitza, 3RO 15I z ekstraktorem Toeplitza, 2RO 8I z ekstraktorem AES-CTR, 3RO 15I z ekstraktorem AES-CTR w neoTRNG. Testy przeprowadzono na płycie deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

Uwaga: Minimalny wskaźnik zaliczenia dla każdego testu statystycznego, z wyjątkiem testu błędzenia losowego (w wariancie *random excursion / random excursion variant*), wynosi w przybliżeniu 96 dla próby o wielkości 100 sekwencji binarnych [14].

Uwaga: Minimalny wskaźnik zaliczenia dla każdego testu statystycznego, z wyjątkiem testu błędzenia losowego (w wariancie *random excursion / random excursion variant*), wynosi w przybliżeniu 96 dla próby o wielkości 100 sekwencji binarnych [14].

Legenda: sp – liczba zaliczonych podtestów (ang. *subtests passed*).

Legenda: * – test niezaliczony.

Legenda: *Tpz* - Toeplitz.

TABELA 4.16: NIST 800-22 : AES 3RO 3-5-7 / Toeplitz 3RO 3-5-7 / AES 2RO 3-5 / Toeplitz 2RO 3-5

Statistical Test	AES 3RO 3-5-7		Tpz 3RO 3-5-7		AES 2RO 3-5		Tpz 2RO 3-5	
Frequency	98/100		98/100		100/100		99/100	
BlockFrequency	100/100		100/100		99/100		100/100	
CumulativeSums	99/100		98/100		100/100		98/100	
CumulativeSums	98/100		99/100		99/100		97/100	
Runs	99/100		99/100		99/100		100/100	
LongestRun	98/100		99/100		98/100		99/100	
Rank	98/100		98/100		99/100		99/100	
FFT	97/100		99/100		99/100		99/100	
NonOverlappingTemplate	148/148	sp	148/148	sp	146/148	sp	148/148	sp
OverlappingTemplate	98/100		100/100		100/100		100/100	
Universal	99/100		98/100		100/100		99/100	
ApproximateEntropy	99/100		100/100		99/100		97/100	
RandomExcursions	8/8	sp	8/8	sp	8/8	sp	8/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	99/100		99/100		100/100		97/100	
Serial	98/100		99/100		100/100		100/100	
LinearComplexity	100/100		100/100		99/100		100/100	

4.3.2 NIST SP 800-90B

Analiza losowości przy użyciu testów NIST SP 800-90B dla konfiguracji z 2RO 8I z ekstraktorem Toeplitza, 3RO 15I z ekstraktorem Toeplitza, 2RO 8I z ekstraktorem AES-CTR, 3RO 15I z ekstraktorem AES-CTR w neoTRNG. Testy przeprowadzono na płytce deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

TABELA 4.17: NIST 800-90 : AES 3RO 3-5-7 / Toeplitz 3RO 3-5-7 / AES 2RO 3-5 / Toeplitz 2RO 3-5

Konfiguracja	H_{min}
AES 3RO 3-5-7	7.487619
Toeplitz 3RO 3-5-7	7.363312
AES 2RO 3-5	7.487618
Toeplitz 2RO 3-5	7.487618

Legenda: $H_{min} = \text{Min}(H_{original}, 8X H_{bitstring})$.

4.3.3 ENT

Analiza losowości przy użyciu testu ENT dla konfiguracji z 2RO 8I z ekstraktorem Toeplitza, 3RO 15I z ekstraktorem Toeplitza, 2RO 8I z ekstraktorem AES-CTR, 3RO 15I z ekstraktorem AES-CTR w neoTRNG. Testy przeprowadzono na płytce deweloperskiej Cora Z7-07S z układem FPGA xc7z007sclg400-1.

TABELA 4.18: ENT : AES 3RO 3-5-7 / Toeplitz 3RO 3-5-7 / AES 2RO 3-5 / Toeplitz 2RO 3-5

Konfiguracja	Entropy
AES 3RO 3-5-7	7.999998
Toeplitz 3RO 3-5-7	7.999998
AES 2RO 3-5	7.999998
Toeplitz 2RO 3-5	7.999998

Legend: Entropy = X bits per byte.

4.4 Wydajność neoTRNG w zależności od typu ekstrakcji

Ostatecznie została przeprowadzona ocena wydajności każdej z konfiguracji generatora w warunkach eksperymentalnych. W trakcie eksperymentu wykorzystano płytkę deweloperską Cora Z7-07S z układem FPGA xc7z007sclg400-1. W celu wyeliminowania wąskiego gardła, jakim w pierwotnym projekcie była transmisja danych losowych po interfejsie UART, przygotowano zmodyfikowane wersje badanych modułów. Z procesu ekstrakcji danych usunięto standardową transmisję UART, zastępując ją licznikiem inkrementowanym przy każdym wygenerowaniu nowego bloku danych. Licznik ten był odczytywany raz na sekundę za pośrednictwem dedykowanego interfejsu UART, po czym następował jego reset. Rzeczywista wydajność została wyznaczona jako iloczyn wartości odczytanej z licznika oraz szerokości wyjściowego bloku danych.

Poniższa tabela Tab. 4.19 przedstawia zestawienie uzyskanych wyników pomiarowych.

TABELA 4.19: Porównanie przepustowości dla różnych konfiguracji oraz metod post-processingu.

Konfiguracja / Metoda	zmierzona wydajność	notatka
FALSE/FALSE	15.625.000 B/s	Wynik zgodny z założeniami $B_{\text{rate}} = \frac{125 \cdot 10^6 \text{ Hz} \cdot 1 \text{ B}}{8} = 15.625.000 \text{ B/s}$ jeden bajt danych na 8 cykli zegara.
FALSE/TRUE	1.923.077 B/s	Wynik zbliżony do założeń → $B_{\text{rate}} = \frac{125 \cdot 10^6 \text{ Hz} \cdot 1 \text{ B}}{64} = 1.953.125 \text{ B/s}$ → jeden bajt danych na 64 cykle zegara.
TRUE/FALSE	~4.182.200 B/s	Wynik średni z kilku pomiarów niedeterministyczne zachowanie vN, brak możliwości wyprowadzenia wyniku
TRUE/TRUE	~471.903 B/s	Wynik średni z kilku pomiarów niedeterministyczne zachowanie vN, brak możliwości wyprowadzenia wyniku
Toeplitz	7.812.496 B/s	Wynik zgodny z założeniami → $B_{\text{rate}} = \frac{125 \cdot 10^6 \text{ Hz} \cdot 16 \text{ B}}{256} = 7.812.496 \text{ B/s}$ → 16 bajtów danych na 256 cykli zegara.
AES	153.797.440 B/s	Wynik niższy niż oczekiwany → $B_{\text{rate}} = \frac{125 \cdot 10^6 \text{ Hz} \cdot 16 \text{ B}}{10} = 200.000.000 \text{ B/s}$ 16 bajtów na 10 cykli zegara. [6] Powód: dodatkowe cykle wykorzystane na pracę generatora i obsługę maszyny stanu.

W ostatniej kolumnie tabeli Tab. 4.19 przedstawiono obliczenia spodziewanej wydajności oraz stwierdza, czy eksperyment pokrył się z założeniami. Wartość $125 \cdot 10^6 \text{ Hz}$ odnosi się do częstotliwości taktowania układu, natomiast pozostałe parametry (np. 1B/8 cykli czy 16B/256 cykli) wynikają bezpośrednio z architektury danego modułu i liczby cykli zegara wymaganych do przetworzenia bloku danych.

W przypadku ekstraktorów opartych na ekstraktorze Von Neumanna teoretyczne wyliczenie przepustowości jest obarczone dużym marginesem błędu ze względu na jego stratny charakter. Koryktor eliminuje pary bitów o tej samej wartości, co sprawia, że faktyczna liczba bitów wyjściowych zależy od charakterystyki statystycznej wejściowego szumu. Z tego względu wiersze zawierające ekstraktor Von Neumanna (TRUE/FALSE) oraz (TRUE/TRUE) nie prezentują konkretnych obliczeń oczekiwanej wydajności.

Należy podkreślić, że ekstrakcja bez ekstraktora Von Neumanna bądź z ekstraktorem Toeplitza oraz AES-CTR ma charakter bezstratny. Można w ich przypadku oczekiwać deterministycznej wydajności na wyjściu, zależnej od rozdzielczości i zaimplementowanego potoku obliczeniowego.

Rozdział 5

Podsumowanie i wnioski

Niniejsza praca poświęcona była analizie sprzętowego generatora liczb losowych neoTRNG. W ramach przeprowadzonych badań eksperymentalnych oceniono wpływ kluczowych parametrów, metod przetwarzania końcowego oraz fizycznego rozmieszczenia elementów w strukturze układu na jakość i charakterystykę statystyczną generowanej losowości.

Analiza danych doświadczalnych uzyskanych z pakietów testowych NIST SP 800-22, NIST SP 800-90B oraz ENT pozwoliła sformułować najistotniejsze wnioski, których uwzględnienie jest kluczowe przed dokonaniem adaptacji generatora neoTRNG w docelowym systemie. Analiza ta została ujęta w czterech kolejnych podrozdziałach.

5.1 Wpływ liczby oscylatorów i liczby inwerterów

Czy skalowanie zasobów sprzętowych poprzez zwiększanie liczby RO oraz liczby inwerterów ma kluczowe znaczenie dla jakości źródła entropii?

- Na podstawie Tab. 4.1, Tab. 4.2, Tab. 4.3 nie można stwierdzić, że konfiguracja 3RO 21I osiągnęła lepsze wyniki niż konfiguracje mniejsze.
- Na podstawie Tab. 4.4 oraz Tab. 4.5 można stwierdzić, że konfiguracja 3RO 21I osiągnęła lepsze wyniki niż konfiguracje mniejsze (przy wyłączonym wielomianie CRC).

Wyniki uzyskane przy użyciu pakietów NIST 800-90 (tab. 4.4) pozwalają uchwycić fakt, że zwiększenie liczby inwerterów w układzie przekłada się na poziom entropii, jaki zapewnia neoTRNG. Zwiększenie entropii układu nie zwalnia jednak projektanta z konieczności zastosowania metod przetwarzania końcowego, co zostało wykazane w tab. 4.2.

5.2 Efektywność przetwarzania końcowego von Neumanna oraz CRC

Czy zastosowanie ekstraktora von Neumanna i wielomianu CRC w module neoTRNG ma znaczenie dla jakości projektowanego generatora?

- Na podstawie tabel Tab. 4.1, Tab. 4.2, Tab. 4.3, Tab. 4.6, Tab. 4.7, Tab. 4.8, Tab. 4.9, Tab. 4.10, Tab. 4.11 można stwierdzić, że wyniki zebrane dla konfiguracji (*FALSE/FALSE*) dyskwalifikują generator w testach statystycznych NIST SP 800-22.
- Na podstawie tabel Tab. 4.1, Tab. 4.2, Tab. 4.3, Tab. 4.6, Tab. 4.7, Tab. 4.8, Tab. 4.9, Tab. 4.10, Tab. 4.11 można stwierdzić, że najlepsze wyniki osiągnęły konfiguracje w kolejności (*TRUE/TRUE*), (*FALSE/TRUE*), (*TRUE/FALSE*) oraz (*FALSE/FALSE*)

Cenną obserwacją jest fakt, że deaktywacja korektora von Neumanna jako ekstraktora losowości przy jednoczesnym aktywnym CRC zapewnia akceptowalny poziom losowości bitstreamu zwracanego przez neoTRNG.

5.3 Wpływ rozmieszczenia RO

Czy rozmieszczenie RO w strukturze FPGA ma istotne znaczenie dla jakości źródła?

- Na podstawie tabel Tab. 4.6, Tab. 4.7, Tab. 4.8, Tab. 4.9, Tab. 4.10, Tab. 4.11, Tab. 4.12, Tab. 4.13, Tab. 4.14, Tab. 4.15 nie można stwierdzić, że rozmieszczenie RO w układzie ma wpływ na losowość.

W celu zarejestrowania istotnych zależności między rozmieszczeniem RO a zmianami w konfiguracji neoTRNG, należałoby w przyszłości podjąć badania nad wpływem bardziej złożonego rozmieszczenia w całej matrycy, niż to zaproponowane w pracy. W przypadku rozproszonego rozmieszczenia RO, deaktywacja korektora von Neumanna przy jednoczesnej aktywacji CRC gwarantuje akceptowalny poziom losowości neoTRNG.

5.4 Zastosowanie zaawansowanych metod przetwarzania końcowego

Czy zaawansowane metody przetwarzania końcowego mają przewagę nad metodami bazowymi?

- Na podstawie Tab. 4.16, Tab. 4.17, Tab. 4.18 nie można stwierdzić przewagi żadnej z konfiguracji ekstraktorów AES-CTR czy Toeplitza. Każda z konfiguracji AES-CTR i Toeplitz przeszła testy.
- Na podstawie Tab. 3.2 oraz Tab. 4.19 można stwierdzić, że AES-CTR umożliwia znaczne zwiększenie wydajności kosztem zasobów bez utraty jakości źródła.
- Na podstawie Tab. 3.2 oraz Tab. 4.19 można stwierdzić, że ekstraktor Toeplitza stanowi kompromis między bazowymi metodami a AES-CTR.
- Generatory wyposażone w AES-CTR lub ekstraktor Toeplitza osiągnęły o wiele większą wydajność w porównaniu z konfiguracją (*TRUE/TRUE*) przy podobnych wynikach testów.

Jeżeli w matrycy FPGA zaimplementowany zostanie układ wymagający znacznej ilości zasobów sprzętowych, ekstraktor Toeplitza jest adekwatnym wyborem. Niemniej jednak, ze względu na silny algorytm kryptograficzny będący podstawą ekstraktora AES-CTR, zaleca się taką implementację układu, aby możliwe było zastosowanie właśnie tego ekstraktora losowości.

5.5 Podsumowanie końcowe

Współczesne systemy brzegowe muszą zapewnić wysoki poziom bezpieczeństwa danych. Konieczne staje się zintegrowanie całego procesu kryptograficznego, którego jednym z podstawowych modułów jest generator liczb losowych. Jego błędna implementacja lub nieumiejętna adaptacja, np. z całkowitym pominięciem ekstraktorów, może skutkować niskim poziomem losowości obserwowanym w wynikach testów NIST. W przypadku celowej implementacji parzystej liczby inwerterów w generatorze neoTRNG można wywołać sytuację zatrzymania procesu kryptograficznego. Sytuacja taka może wystąpić, jeżeli zastosowany został ekstraktor losowości w formie korektora von Neumanna. Jest to istotna obserwacja, na którą dokumentacja projektu w serwisie GitHub nie zwraca uwagi.

Projektowanie systemów brzegowych realizowanych z użyciem matryc FPGA wymaga często wykorzystania większości ich zasobów. Tym samym może pojawić się konieczność redukcji nadmiarowych zasobów sprzętowych w tych modułach, w których jest to potencjalnie możliwe. Naturalnie takim modułem może być generator liczb losowych oparty na RO. Wykazano, że kluczem do optymalnego zaprojektowania układu TRNG nie jest wyłącznie maksymalizacja liczby oscylatorów, lecz balans pomiędzy: dostrojeniem wykorzystania zasobów, rozmieszczeniem fizycznym elementów w układzie FPGA, odpowiednim doбором ekstraktora oraz rygorystycznym przeprowadzeniem testów. Przedstawione w ramach pracy wyniki eksperymentalne oraz wnioski końcowe stanowią pogłębione studium nad adaptacją generatora neoTRNG w rozwiązaniach OSHW, cechujących się dużym wykorzystaniem zasobów sprzętowych podczas implementacji.

Literatura

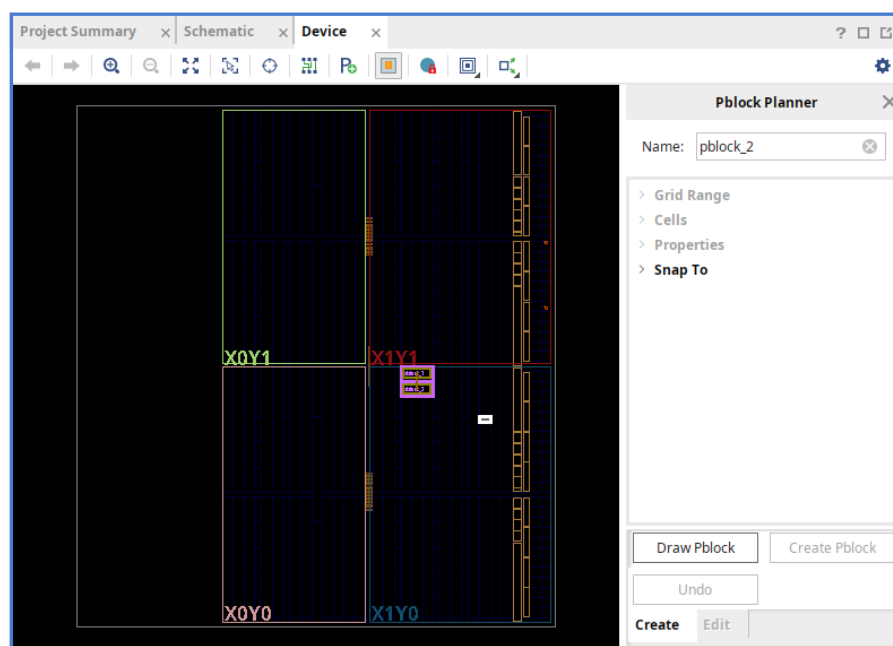
- [1] Anastasios Bikos, Panagiotis E. Nastou, Georgios Petroudis, and Yannis C. Stamatou. Random number generators: Principles and applications. *MDPI*, 2023.
- [2] Morris Dworkin. Nist sp 800-38a recommendation for block cipher modes of operation: Methods and techniques. *NIST*, 2023.
- [3] Quantum Flagship. Quantum random numbers generator. <https://qt.eu/quantum-principles/sensing-and-metrology/qrng>.
- [4] Fourmilab. Ent: A pseudorandom number sequence test program. <https://www.fourmilab.ch/random/>.
- [5] Maciej Gajdzica. Jak działa crc. <https://ucgosu.pl/2017/01/jak-dziala-crc/>, 2017.
- [6] Hosein Hadipour. Aes-vhdl-hosein. <https://hoseinhadipour.com/AES-VHDL/>.
- [7] Zeshan Haider, Muhammad Haroon Saeed, Muhammad Ehsan ul Haq Zaheer, Zeeshan Ahmed Alvi, Muhammad Ilyas, Tahira Nasreen, Muhammad Imran, Rameez Ul Islam, and Manzoor Ikram. Quantum random number generator (qrng): Theoretical and experimental investigations. *arxiv*, 2025.
- [8] Peter Kok. A first introduction to quantum physics. *Springer Nature Link*, 2023.
- [9] PPhong Q. Nguyen and Igor E. Shparlinaki. The insecurity of the elliptic curve digital signature algorithm with partially known nonces. *Springer Nature Link*, 2003.
- [10] Colin O’Flynn. Experimenting with metastability and multiple clocks on fpgas. 2020.
- [11] Dmytro Proskurin, Maksim Iavich, Tetiana Okhrimenko1, Okoro Chukwukaelonma1, and Tetiana Hryniuk1. Predicting pseudo-random number generator output with sequential analysis. *CEUR Workshop Proceedings*, 2024.
- [12] ID Quantique. Quantis when random numbers cannot be left to chance. https://lenlasers.ru/upload/iblock/c27/Quantis-QRNG_Brochure.pdf.
- [13] Jan M. Rabaey, Borivoje Nikolic, and Anantha P. Chandrakasan. *Digital Integrated Circuits: A Design Perspective*. 1995.
- [14] Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, N. Heckert, James Dray, San Vo, and Lawrence Bassham. Nist sp 800-22: A statistical test suite for random and pseudorandom number generators for cryptographic applications. *NIST Information Technology Laboratory*, 2010.
- [15] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. 2016.
- [16] Juan Soto. Nist ir 6390 randomness testing of the advanced encryption standard candidate algorithms. *NIST*, 1999.
- [17] Lakshmi Sreekumar and P. Ramesh. Selection of an optimum entropy source design for a true random number generator. *Procedia Technology*, 2016.

- [18] stnolting. neotrng: A true random number generator for fpgas.
<https://github.com/stnolting/neoTRNG>, 2016.
- [19] Meltem Sönmez Turan, Elaine Barker, John Kelsey, Kerry McKay, Mary Baish, and Michael Boyle. Nist sp 800-90b: Recommendation for the entropy sources used for random bit generation. *NIST Information Technology Laboratory*, 2018.
- [20] Salil P. Vadhan. Pseudorandomness survey. *Harvard University Cambridge*, 2016.
- [21] Scott Vanstone, Alfred J. Menezes, and Paul van Oorschot. Handbook of applied cryptography. *CRC Press*, 1996.
- [22] Benjamin Williams, Robert E. Hiromoto, and Albert Carlson. Analysis of a cryptographically secure pseudo random number generator. *IEEE*, 2023.
- [23] Xiaoguang Zhang, You-Qi Nie, Hao Liang, and Jun Zhang. Fpga implementation of toeplitz hashing extractor for real time post-processing of raw random numbers. *IEEE*, 2016.
- [24] Rok Zitko. Toeplitz entropy extractor. <https://github.com/rokzitko/toeplitz>.

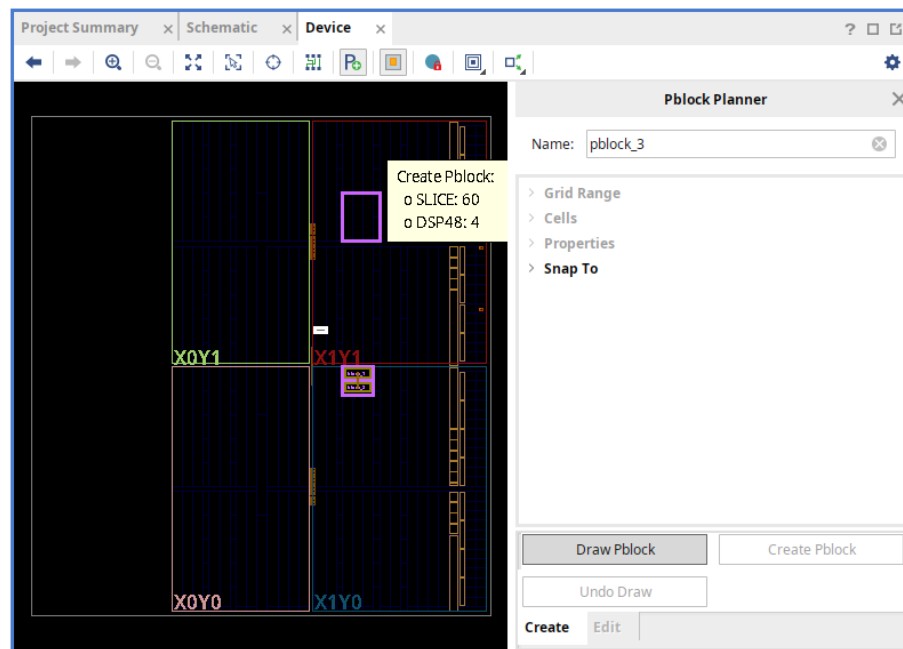
Dodatek A

Floorplanning

Analiza rozmieszczenia została wykonana za pomocą narzędzia do planowania rozmieszczenia logiki wewnątrz układu scalonego (ang. *floorplanning*). Możliwość ręcznego modyfikowania położenia fragmentów układu scalonego dostępna jest w narzędziu Vivado po wybraniu opcji: RTL Analysis → Open Elaborated Design, a następnie po zmianie widoku z Default Layout na Floorplanning. Po wybraniu opcji Show Pblock Planner View dostępne jest okno dialogowe:

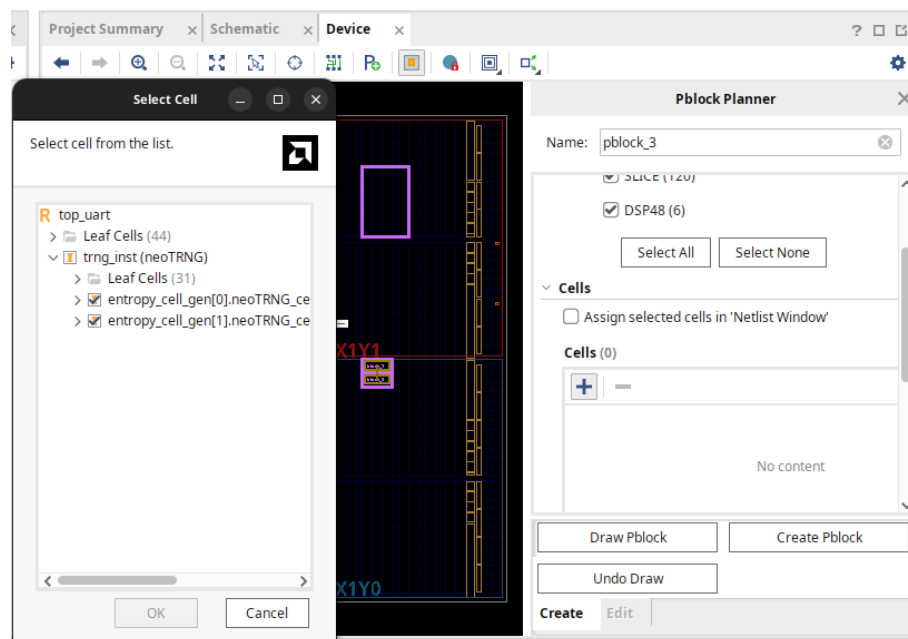


RYSUNEK A.1: Okno dialogowe po prawej stronie



RYSUNEK A.2: Ręczne wyznaczanie granic Pblock

Okno to umożliwia tworzenie, usuwanie oraz edycję elementów typu Pblock. Pblock jest obiektem definiującym granice obszaru, w którym mogą zostać umieszczone wybrane elementy struktury.



RYSUNEK A.3: Wybór elementów układu mających znaleźć się w obrębie Pblock

Po wyborze elementów należy wybrać opcję Create Pblock. Informacje o utworzonym Pblock znajdują się w Sources/Constraints/constrs_1/pblocks.xdc

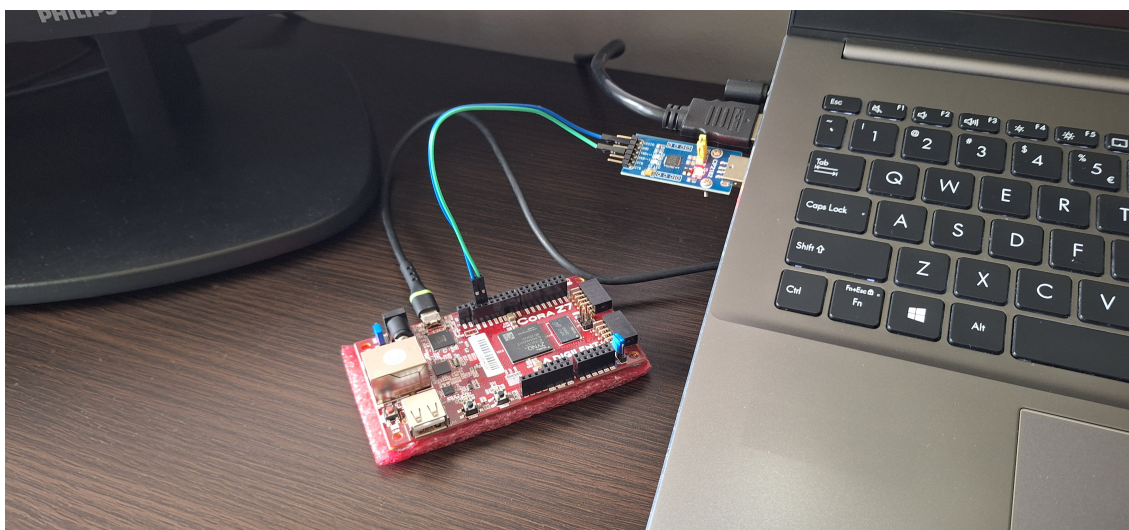
Dodatek B

Automatyzacja pomiarów i testów

B.1 Wymagania sprzętowe

Sprzęt konieczny do przeprowadzenia badań:

- dowolna płytki deweloperska z układem FPGA,
- konwerter UART/USB,
- komputer PC z systemem operacyjnym Linux.



RYSUNEK B.1: Aparatura pomiarowa

B.2 Zbieranie danych

Użyteczne komendy:

```
sudo stty -F /dev/ttyUSB0 921600 raw -echo
```

Konfiguracja prędkości (baudrate) portu szeregowego.

```
sudo dd if=/dev/ttyUSB0 of=<file>.bin bs=1M count=100 status=progress iflag=fullblock
```

Zebranie 100 MiB danych losowych do pliku binarnego.


```
sudo xxd /dev/ttyUSB0
```

Podgląd danych na żywo.

```
sudo cat /dev/ttyUSB0 | pv -r > /dev/null
```

Licznik prędkości (baudrate) na żywo ([x]B/s).

B.3 Uruchamianie pojedynczych testów

```
<ENT dir>$ ./ent <file>.bin
```

Uruchomienie ENT.

```
<NIST SP 800-22 dir>$ ./assess 1000000
```

Uruchomienie NIST SP 800-22.

```
<NIST SP 800-90B dir>$ ./ea_non_iid <file>.bin 8
```

Uruchomienie NIST SP 800-90B.

```

1  $ ./assess 1000000
2      G E N E R A T O R      S E L E C T I O N
3      -----
4
5      [0] Input File           [1] Linear Congruential
6      [2] Quadratic Congruential I   [3] Quadratic Congruential II
7      [4] Cubic Congruential       [5] XOR
8      [6] Modular Exponentiation    [7] Blum-Blum-Shub
9      [8] Micali-Schnorr           [9] G Using SHA-1
10
11  Enter Choice: 0
12
13
14      User Prescribed Input File: <file>.bin
15
16      S T A T I S T I C A L   T E S T S
17      -----
18
19      [01] Frequency           [02] Block Frequency
20      [03] Cumulative Sums     [04] Runs
21      [05] Longest Run of Ones [06] Rank
22      [07] Discrete Fourier Transform [08] Nonperiodic Template Matchings
23      [09] Overlapping Template Matchings [10] Universal Statistical
24      [11] Approximate Entropy [12] Random Excursions
25      [13] Random Excursions Variant [14] Serial
26      [15] Linear Complexity
27
28      INSTRUCTIONS
29      Enter 0 if you DO NOT want to apply all of the
30      statistical tests to each sequence and 1 if you DO.
31
32  Enter Choice: 1
33
34      P a r a m e t e r   A d j u s t m e n t s
35      -----
36      [1] Block Frequency Test - block length(M):      128
37      [2] NonOverlapping Template Test - block length(m): 9
38      [3] Overlapping Template Test - block length(m):  9
39      [4] Approximate Entropy Test - block length(m):   10
40      [5] Serial Test - block length(m):                16
41      [6] Linear Complexity Test - block length(M):     500
42
43  Select Test (0 to continue): 0
44
45  How many bitstreams? 100
46
47  Input File Format:
48      [0] ASCII - A sequence of ASCII 0's and 1's
49      [1] Binary - Each byte in data file contains 8 bits of data
50
51  Select input mode: 1
52
53      Statistical Testing In Progress.....

```

B.4 Struktura zbioru danych

```

data/1_testy_bazowe/2RO_8I/$ tree
├── false_false.bin
├── false_true.bin
├── true_false.bin
├── true_true.bin
├── tests_results
│   ├── ent
│   │   ├── false_false.log
│   │   ├── false_true.log
│   │   ├── true_false.log
│   │   └── true_true.log
│   ├── nist22
│   │   ├── false_false.txt
│   │   ├── false_true.txt
│   │   ├── true_false.txt
│   │   ├── true_true.txt
│   │   └── table.txt
│   └── nist90
│       ├── false_false.log
│       ├── false_true.log
│       ├── true_false.log
│       └── true_true.log
└── utilization
    ├── false_false.rpt
    ├── false_true.rpt
    ├── true_false.rpt
    └── true_true.rpt

```

RYSUNEK B.3: Struktura katalogów dla konfiguracji 2RO_8I.

Dla każdej z badanych konfiguracji RO powstał wynikowy katalog zgodny ze schematem (Fig. B.3):

- **root** – przechowuje pliki binarne z zebranymi danymi,
- **tests_results** – katalog na wyniki wszystkich testów,
- **ent** – wyniki testów ENT,
- **nist22** – wyniki testów NIST SP 800-22 oraz wygenerowana tabela,
- **nist90** – wyniki testów NIST SP 800-90B,
- **utilization** – raporty wykorzystania zasobów.

Łącznie w ramach badań zebrano:

- 40 plików .bin (łącznie około 4 GB danych),

- 129 plików z wynikami testów .log i .txt,
- 40 raportów wykorzystania zasobów.

B.5 Automatyzacja testów

Wykonanie testów zostało zautomatyzowane za pomocą skryptów w języku Python:

- `ent_automation.py` – przyjmuje ścieżkę do katalogu `root` z badaną konfiguracją, wyszukuje pliki `.bin`, uruchamia testy i zapisuje wyniki w `root/tests_results/ent/`,
- `nist22_automation.py` – przyjmuje ścieżkę do katalogu `root` z badaną konfiguracją, wyszukuje pliki `.bin`, uruchamia testy i zapisuje wyniki w `root/tests_results/nist22/`,
- `nist90_automation.py` – przyjmuje ścieżkę do katalogu `root` z badaną konfiguracją, wyszukuje pliki `.bin`, uruchamia testy i zapisuje wyniki w `root/tests_results/nist90/`,
- `resource_automation.py` – przyjmuje ścieżkę do katalogu `data` ze wszystkimi folderami różnych konfiguracji, wyszukuje pliki `.rpt` i zestawia wyniki w jedną tabelę $\text{\LaTeX}$,
- `table_automation.py` – przyjmuje ścieżkę do katalogu `nist22` z badaną konfiguracją, wyszukuje wyniki testów NIST SP 800-22 i zestawia je w formę tabeli $\text{\LaTeX}$.

```

1 <path>/data$ python3 ent_automation.py ./1_testy_bazowe/2RO_8I/
2 ENT processing false_false.bin...
3 Entropy = 5.322542 bits per byte.
4 Saved -> <path>/data/1_testy_bazowe/2RO_8I/tests_results/ent/false_false.log
5 ENT processing false_true.bin...
6 Entropy = 7.999742 bits per byte.
7 Saved -> <path>/data/1_testy_bazowe/2RO_8I/tests_results/ent/false_true.log
8 ENT processing true_false.bin...
9 Entropy = 7.168778 bits per byte.
10 Saved -> <path>/data/1_testy_bazowe/2RO_8I/tests_results/ent/true_false.log
11 ENT processing true_true.bin...
12 Entropy = 7.999997 bits per byte.
13 Saved -> <path>/data/1_testy_bazowe/2RO_8I/tests_results/ent/true_true.log

```

RYSUNEK B.4: Uruchomienie skryptu automatyzacji testów ENT dla `/1_testy_bazowe/2RO_8I`.



© 2026 Szymon Krzysztof Gogulski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu L^AT_EX.